



中国科学院大学
University of Chinese Academy of Sciences

Deep Learning

Deep Generation Network

Xinfeng Zhang (张新峰)

School of Computer Science and Technology

University of Chinese Academy of Sciences

Email: xfzhang@ucas.ac.cn



计算机科学与技术学院

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY



提纲

- 深度生成模型概述
- Hopfield神经网络
- 玻尔兹曼机和受限玻尔兹曼机
- 深度玻尔兹曼机和深度信念网络
- 自编码器及其变种
- 扩散模型



1

深度生成模型概述

深度生成模型概述

- ❑ 在第五章“生成对抗网络”中已经提到，生成方法通过学习样本的联合概率分布 $P(X,Y)$ 来训练生成模型，以生成符合样本分布的新数据。
- ❑ 生成模型在无监督学习中占据主要位置，可以用于在没有标签的情况下捕捉样本之间的高阶相关性。
- ❑ 与卷积神经网络、循环神经网络相比，深度生成模型是深度学习模型的另外一种结构，也常被表示为图模型。



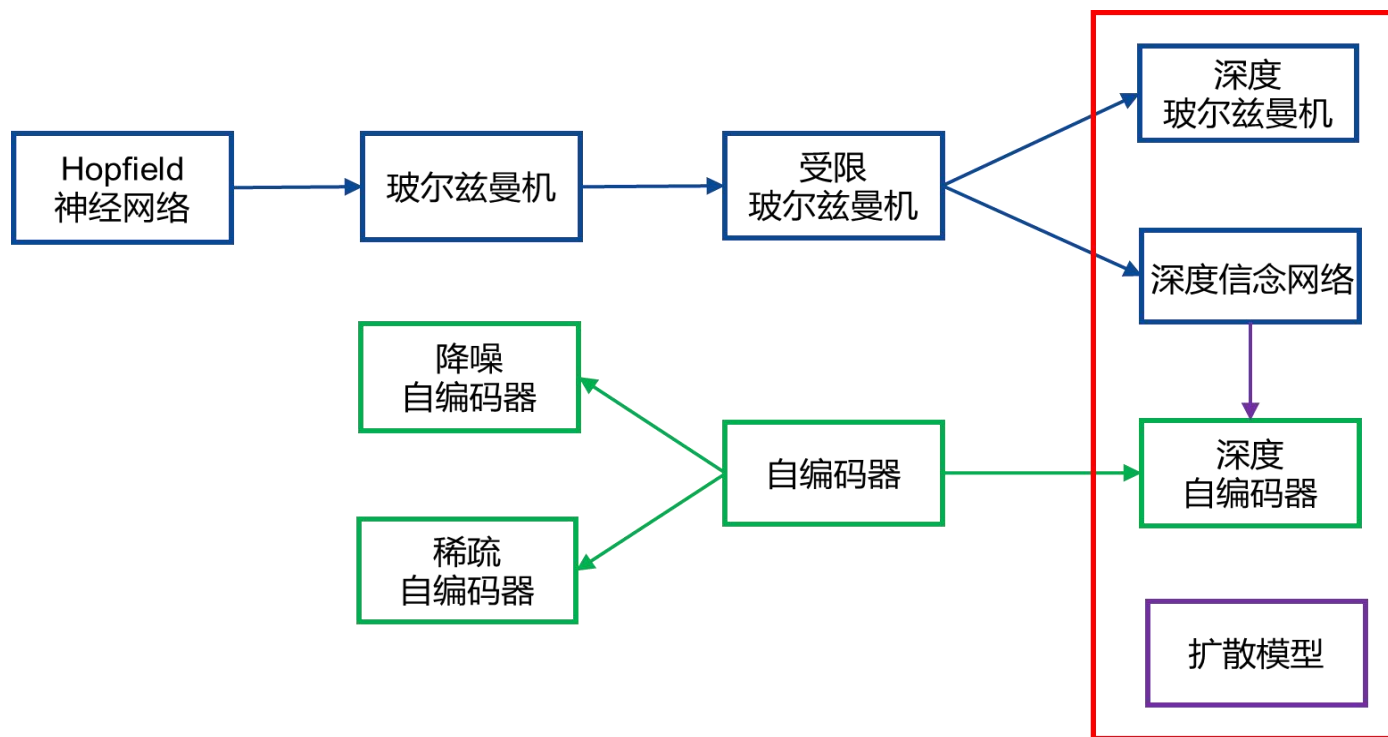
深度生成模型概述

- ❑ Sigmoid信念网（Sigmoid Belief Networks, SBN）就是一种生成式多层神经网络，并采用变分近似的方法进行训练，但是在实际应用中获得多层联合概率分布是非常困难的。
- ❑ 2006年，Geoffrey Hinton等人基于Sigmoid信念网提出了称作深度信念网络（Deep Belief Networks, DBN）的深度生成模型，它与Sigmoid信念网结构不同的地方是最上面两层使用了受限玻尔兹曼机（Restricted Boltzmann Machine, RBM）。
- ❑ 受到这种训练方式的启发，出现了更多种类的深度生成模型，如深度玻尔兹曼机（Deep Boltzmann Machines, DBM）、深度自编码器（Deep Autoencoder, DAE）等。



深度生成模型概述

- 将深度生成模型分为三类：一类是以受限玻尔兹曼机为基础的深度玻尔兹曼机与深度信念网络，一类是以自编码器为基础的深度自编码器，一类是近期出现的扩散模型（Diffusion Model）。



深度生成模型概述

年份	事件	相关论文/Reference
1982	Hopfield网络提出	J. J . Hopfield . Neural networks and physical systems with emergent collective computational abilities. Proceedings of the National Academy of Sciences, Vol. 79 , No. 8, pp. 2554-2558, 1982
1983	玻尔兹曼机生成模型被提出	Fahlman, S. E., Hinton, G. E., & Sejnowski, T. J. (1983). Massively parallel architectures for AI: NETL, Thistle, and Boltzmann machines. Proceedings of AAAI-83109, 113
1992	Neal提出了sigmoid信念网络	Neal, R. M. (1992). Connectionist learning of belief networks. Artificial intelligence, 56(1), 71-113
2006	Hinton提出深度信念网络	Hinton, G. E., Osindero, S., & Teh, Y. W. (2006). A fast learning algorithm for deep belief nets. Neural computation, 18(7), 1527-1554
2008	采用DBNs来完成回归任务	Hinton, G. E., & Salakhutdinov, R. R. (2008). Using deep belief nets to learn covariance kernels for Gaussian processes. In Advances in neural information processing systems (pp. 1249-1256)
2009	将卷积深度信念网络用于可视物体识别	Lee, H., Grosse, R., Ranganath, R., & Ng, A. Y. (2009, June). Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations. In Proceedings of the 26th annual international conference on machine learning (pp. 609-616). ACM
2012	DBNs应用在语音识别领域	Mohamed, A. R., Dahl, G. E., & Hinton, G. (2012). Acoustic modeling using deep belief networks. IEEE Transactions on Audio, Speech, and Language Processing, 20(1), 14-22
2014	Goodfellow提出生成对抗网络	Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). Generative adversarial nets. In Advances in neural information processing systems (pp. 2672-2680)
2016	Deepmind创建了一个深度生成模型Wavenet, 可用来生成具有自然人声的语音	Van Den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., ... & Kavukcuoglu, K. (2016). Wavenet: A generative model for raw audio. arXiv preprint arXiv:1609.03499
2020	OpenAI提出了Diffusion model的图像生成模型	Ho, Jonathan, Ajay Jain, and Pieter Abbeel. "Denoising diffusion probabilistic models." Advances in neural information processing systems 33 (2020): 6840-6851.





2

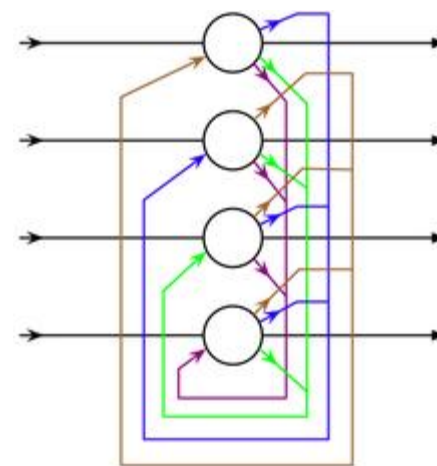
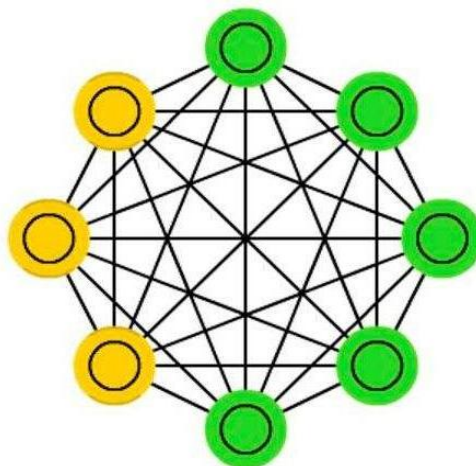
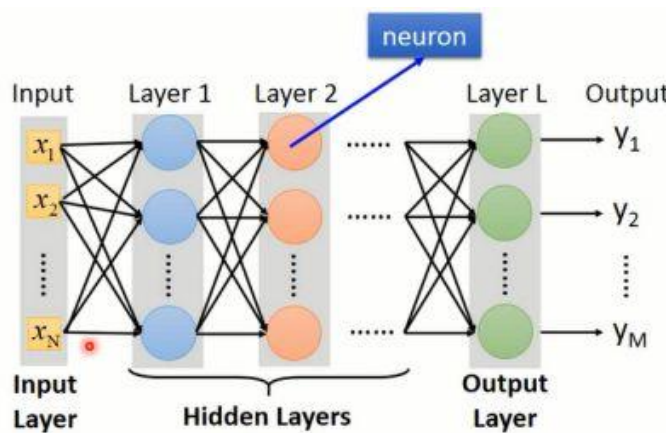
Hopfield神经网络



Hopfield神经网络

□ 神经网络分类

- 多层神经网络：模式识别
- 相互连接型网络：通过联想记忆去除数据中的噪声



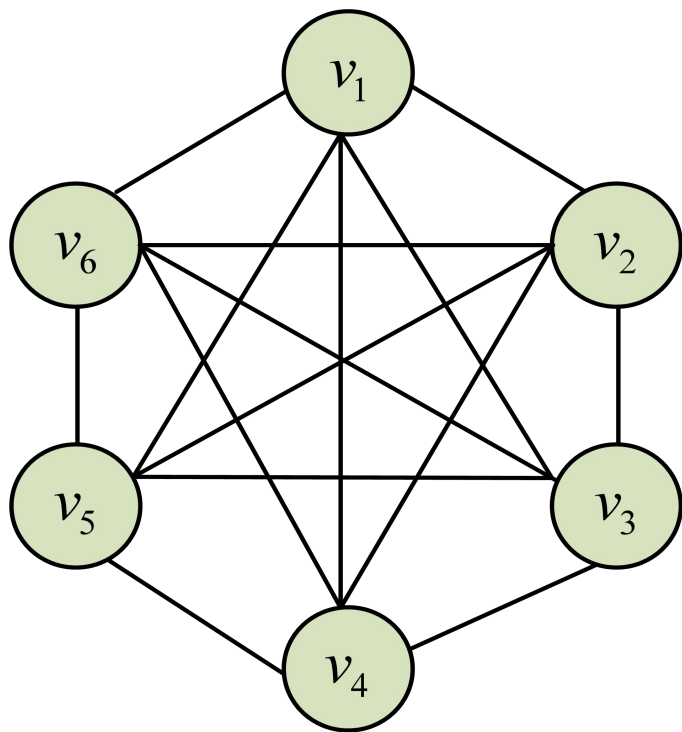
Hopfield神经网络

- 1982年提出的Hopfield神经网络是最典型的相互连结型网络
 - 它是一种相互连接型神经网络，也可以看作是一种单层全连接反馈神经网络。
- 根据激活函数的不同，Hopfield神经网络可分为离散型Hopfield神经网络（Discrete Hopfield Neural Networks, DHNN）和连续型Hopfield神经网络（Continuous Hopfield Neural Networks, CHNN）两种。



Hopfield神经网络

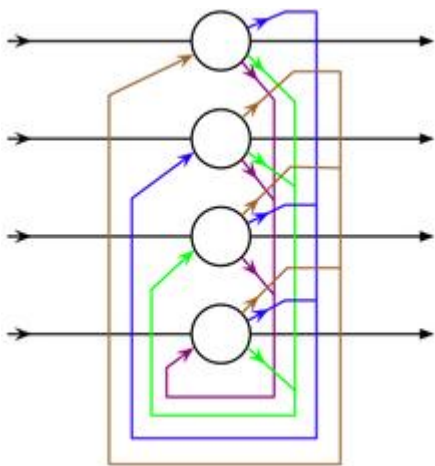
- Hopfield神经网络中的每个神经元都将自己的状态传递给其他所有神经元，同时又接受其他所有神经元传递来的信息。以6个神经元的Hopfield神经网络为例，网络结构如图所示。



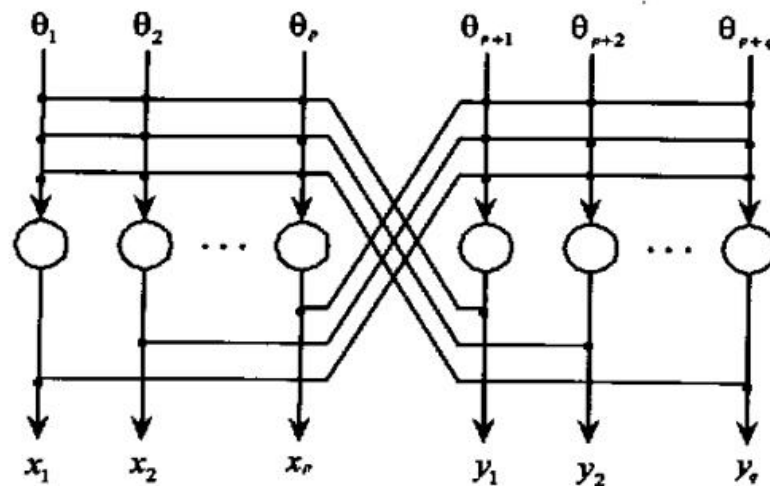
Hopfield神经网络

□ 联想记忆就是当输入模式为某种状态时，输出端要给出与之相应的输出模式

- 如果输入模式与输出模式一致，称为自联想记忆，否则称为异联想记忆



自联想记忆



异联想记忆

输入: $x_1, x_2, \dots, x_p$

输出: $y_1, y_2, \dots, y_q$

Hopfield神经网络

□ Hopfield神经网络计算

- 假设有 n 个单元组成的Hopfield神经网络，第 i 个单元在 t 时刻的输入记作 $u_i(t)$ ，输出记作 $x_i(t)$ ，连接权重为 w_{ij} ，阈值为 $b_i(t)$ ，则 $t + 1$ 时刻 i 单元的输出 $x_i(t + 1)$ 可表示为：

$$x_i(t + 1) = \begin{cases} 1, & u_i(t) > 0 \\ x_i(t), & u_i(t) = 0 \\ 0, & u_i(t) < 0 \end{cases}$$

权重总和大于阈值，取1
权重总和小于阈值，取0

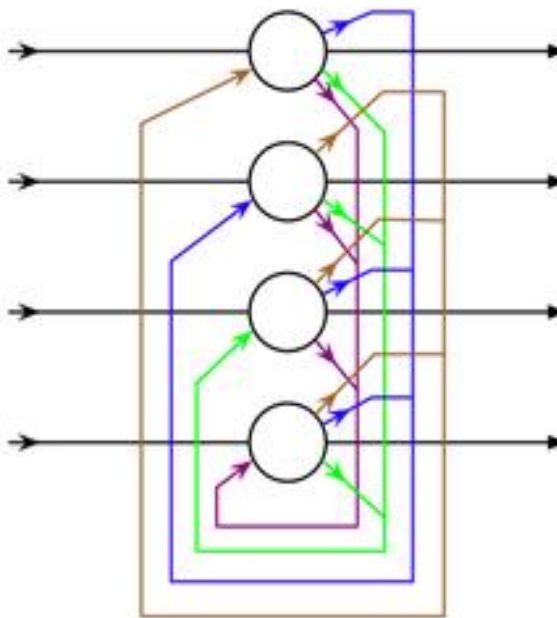
$$u_i(t) = \sum_{j=1}^n w_{ij} x_j(t) - b_i(t)$$

来自于其他单元的输入
 $x_j(t)$ 的权重总和



Hopfield神经网络

- 在Hopfield神经网络中，每个时刻都**只有一个**随机选择的单元会发生状态变化
- 对于一个由 n 个单元组成的网络，如果要完成全部单元的状态变化，至少需要 n 个时刻
 - 单元的状态变化会一直进行下去，直到网络达到稳定状态。各单元的最终状态就是输出模式



Hopfield神经网络

- 根据输入模式联想输出模式时，需要事先确定连接权重 w_{ij} ，而连接权重 w_{ij} 要对输入模式的训练样本进行训练后才能确定
- 和多层神经网络一样，一次训练并不能确定连接权重，而是要不断重复这个过程，直到满足终止判断条件，而这个指标就是Hopfield神经网络的**能量函数 E**

$$E = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} x_i x_j + \sum_{i=1}^n b_i x_i$$

- 当输入模式与输出模式一致时，能量函数 E 的结果是0



Hopfield神经网络

□ 根据前面定义的状态变化规则改变网络状态时，上式中定义的能量函数 E **总是非递增的**，即随时间的不断增加而逐渐减小，直到网络达到稳定状态为止

— 能量函数分解成单元 k 的能量函数和 k 以外的单元的能量函数

$$\begin{aligned} E &= -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} x_i x_j + \sum_{i=1}^n b_i x_i \\ E &= -\frac{1}{2} \left(x_k \sum_{j=1}^n w_{kj} x_j + \sum_{i \neq k} \sum_{j=1}^n w_{ij} x_i x_j \right) + b_k x_k + \sum_{i \neq k} b_i x_i \\ &= -\frac{1}{2} \left(x_k \left(w_{kk} x_k + \sum_{j \neq k} w_{kj} x_j \right) + \sum_{i \neq k} \left(w_{ik} x_i x_k + \sum_{j \neq k} w_{ij} x_i x_j \right) \right) + b_k x_k + \sum_{i \neq k} b_i x_i \\ &= -\frac{1}{2} \left(w_{kk} x_k^2 + x_k \sum_{j \neq k} w_{kj} x_j + \sum_{i \neq k} \left(w_{ik} x_i x_k + \sum_{j \neq k} w_{ij} x_i x_j \right) \right) + b_k x_k + \sum_{i \neq k} b_i x_i \\ &= -\frac{1}{2} \sum_{i \neq k} \sum_{j \neq k} w_{ij} x_i x_j + \sum_{i \neq k} b_i x_i - \frac{1}{2} \left(\sum_{j \neq k} w_{kj} x_j + \sum_{i \neq k} w_{ik} x_i \right) x_k + b_k x_k \end{aligned}$$



Hopfield神经网络

□ 根据前面定义的状态变化规则改变网络状态时，上式中定义的能量函数 E **总是非递增的**，即随时间的不断增加而逐渐减小，直到网络达到稳定状态为止

— 能量函数分解成单元 k 的能量函数和 k 以外的单元的能量函数

$$E = -\frac{1}{2} \sum_{i \neq k} \sum_{j \neq k} w_{ij} x_i x_j + \sum_{i \neq k} b_i x_i - \frac{1}{2} \left(\sum_{j \neq k} w_{kj} x_j + \sum_{i \neq k} w_{ik} x_i \right) x_k + b_k x_k$$

$$\Delta E_k = -\frac{1}{2} \left(\sum_{j \neq k} w_{kj} x_j + \sum_{i \neq k} w_{ik} x_i \right) \Delta x_k + b_k \Delta x_k \quad \Delta x_k = x_k(t+1) - x_k(t)$$

$$\Delta E_k = -\left(\sum_{j \neq k} w_{kj} x_j - b_k \right) \Delta x_k = -u_k \Delta x_k$$

当 $\Delta x_k > 0$ 时， x_k 由 0 变成 1，即 $u_k(t) > 0$

$$\Delta E_k(t) < 0$$

当 $\Delta x_k < 0$ 时， x_k 由 1 变成 0，即 $u_k(t) < 0$

$$\Delta E_k(t) < 0$$

$$x_k(t+1) = \begin{cases} 1, & u_k(t) > 0 \\ x_k(t), & u_k(t) = 0 \\ 0, & u_k(t) < 0 \end{cases}$$



Hopfield神经网络

□ 根据前面定义的状态变化规则改变网络状态时，上式中定义的能量函数 E **总是非递增的**，即随时间的不断增加而逐渐减小，直到网络达到稳定状态为止

— 能量函数分解成单元 k 的能量函数和 k 以外的单元的能量函数

$$u_i(t+1) = \sum_{j=1}^n w_{ij}x_j(t) - b_i(t) \quad E = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij}x_i x_j + \sum_{i=1}^n b_i x_i$$

$$u_i(t+1) = \sum_{j=1}^n w_{ij}x_j(t) + b_i(t) \quad E = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij}x_i x_j - \sum_{i=1}^n b_i x_i$$



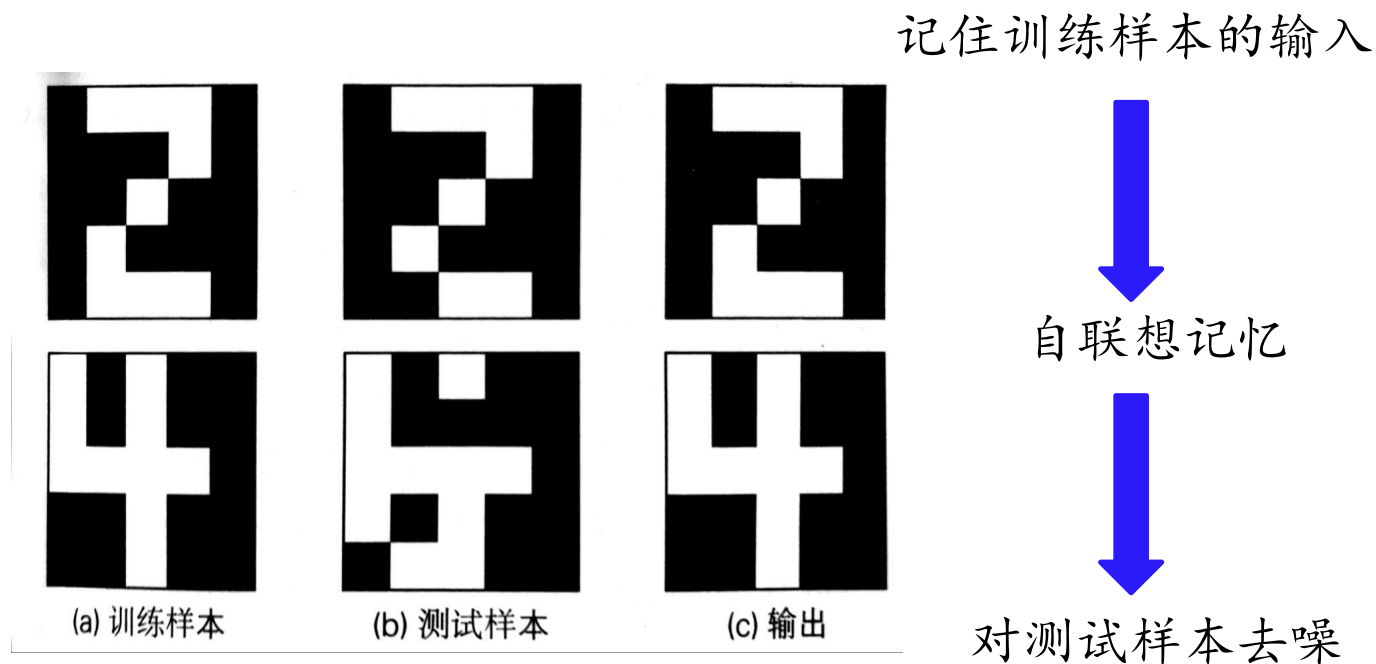
Hopfield神经网络

□ 训练得到权重矩阵后，网络的参数就确定下来，网络的运行流程为：

- 对网络进行初始化，为每个神经元赋初始状态。
- 从网络中随机或按照顺序选取一个神经元。
- 更新该神经元的状态，其他神经元的状态保持不变。
- 求当前状态下网络的能量，判断网络是否达到稳定状态，若达到稳定状态或满足给定条件（如限定迭代次数）则结束；否则转到2）继续运行。



Hopfield神经网络

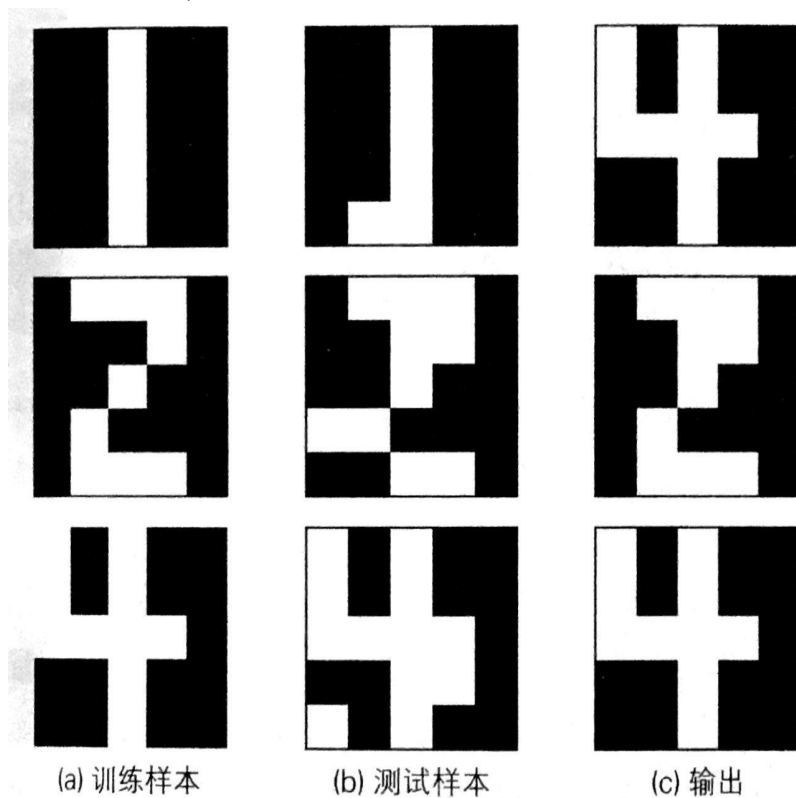


Hopfield网络的应用



Hopfield神经网络

- 当需要记忆的模式之间的较为相似，或者需要记忆的模式太多时，Hopfield神经网络就不能正确地辨别模式。这种相互干扰、不能准确记忆的情况称为 **串扰(crosstalk)**



由于“4”中包含了“1”中的全部竖线，Hopfield网络错把带有“横线”噪音的测试样本“1”识别为“4”，这种情况即串扰

Hopfield网络的串扰



Hopfield神经网络

- Hopfield神经网络能够记忆的模式数量有限，大约是网络单元数的**15%左右**，为了防止串扰，可以采用先把模式正交化再进行记忆等方法
- 但是正交化方法并不能完全解决问题，**玻尔兹曼机**可以解决这一问题





3

玻尔兹曼机及受限玻尔兹曼机



玻尔兹曼机

- 玻尔兹曼机（Boltzmann Machines, BM）通常是指一种随机的离散型Hopfield神经网络，是具有隐单元的全连接反馈神经网络。这种网络在神经元状态变化中引入了统计概率，网络的平衡状态服从玻尔兹曼分布或称吉布斯分布（Gibbs distribution），网络运行机制基于模拟退火算法。

$$p_i \propto e^{-\varepsilon_i/(kT)}$$

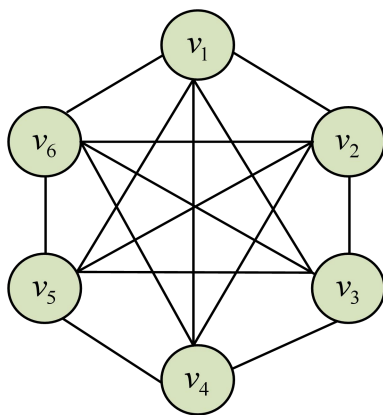
其中 p_i 是系统处于状态 i 的概率， ε_i 是该状态的能量， kT 为玻尔兹曼常数 k 和热力学温度 T 的乘积。符号 $\propto$ 表示比例



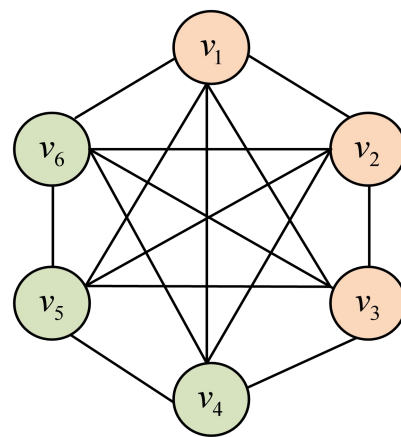
玻尔兹曼机

□ 在Hopfield神经网络中，神经元的功能及其在网络中的地位是一样的，但在玻尔兹曼机中，一部分神经元与外部相连，称为可见单元，完成网络的输入、输出功能，或者严格地说可以受到外部条件的约束；另一部分神经元则不与外部相连，称作隐藏单元，在训练中起辅助作用。

- 图展示的是一个包含6个神经元的玻尔兹曼机，其中 v_1 、 v_2 和 v_3 是隐藏单元， v_4 、 v_5 和 v_6 是可见单元。



Hopfield神经网络



玻尔兹曼机

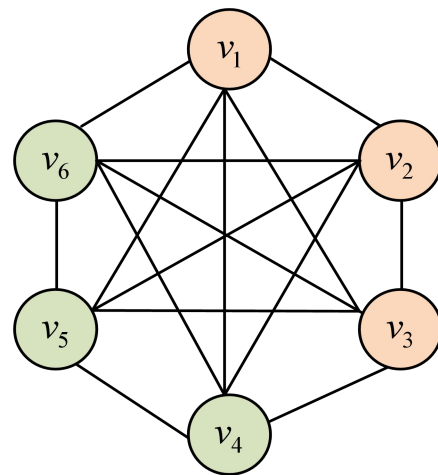
玻尔兹曼机

□ 玻尔兹曼机也是相互连接型网络

- 如果发生串扰或陷入局部最优解，Hopfield神经网络就不能正确的辨别模式。而玻尔兹曼机(Boltzmann Machine)则可以通过让每个单元按照一定的概率分布发生状态变化，来避免陷入局部最优解

□ 玻尔兹曼机保持了Hopfield神经网络的假设：

- 权重对称
- 自身无连接
- 二值输出



玻尔兹曼机

玻尔兹曼机

- 玻尔兹曼机中，神经元的状态为0或1的概率则取决于相应的输入。具体而言，单个神经元 j 的净输入为：

$$net_j = \sum_i w_{ij}x_i - b_j$$

- 净输入并不能通过符号转移函数直接获得确定的输出状态，实际的输出状态将按照某种概率发生，输出某种状态的转移概率。神经元 j 的状态为1的概率为：

$$P_j(1) = \frac{1}{1 + e^{-net_j/T}}$$

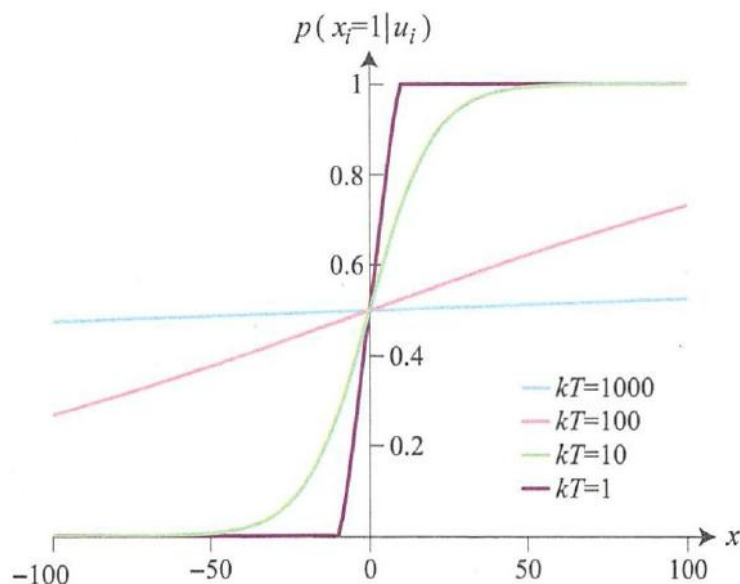
- 其中 T 为温度系数。温度系数较大时，获得能量函数最小值的概率较低。反之，温度系数较小时，虽然获得能量函数最小值的概率增加了，但是玻尔兹曼机需要较长时间才能达到稳定状态。玻尔兹曼机的训练选择模拟退火算法（Simulated Annealing）。为了避免上述情况的出现，可以先采用较大的温度系数进行粗调，然后逐渐减小温度系数进行微调。



玻尔兹曼机

□ 玻尔兹曼机的输出是按照某种概率分布决定的

- $T (>0)$ 表示温度系数，当 T 趋近于无穷时，无论 u_i 取值如何， x_i 等于 1 或 0 的概率都是 1/2
- k 是玻尔兹曼常数



温度系数引起的概率变化

玻尔兹曼机

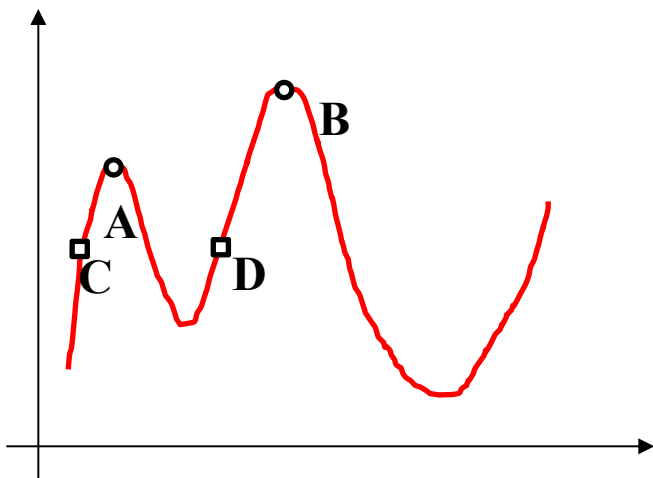
□ 玻尔兹曼机选择**模拟退火算法**，可以先采用较大的温度系数进行粗调，然后逐渐减小温度系数进行微调

- 温度系数越大，跳出局部最优解的概率越高。但是温度系数增大时，获得能量函数极小值的概率就会降低
- 反之，温度系数减小时，虽然获得能量函数极小值的概率增加了，但是玻尔兹曼机需要经历较长时间才能达到稳定状态



玻尔兹曼机

□ 模拟退火算法



模拟退火是一种贪心算法，但是它的搜索过程引入了随机因素，以一定的概率来接受一个比当前解要差的解，因此有可能会跳出这个局部的最优解，达到全局的最优解，而且这个概率随着时间推移逐渐降低(逐渐降低才能趋向稳定)



玻尔兹曼机

□ 模拟退火算法

– 示例: 求函数 $F(x)=6x^2+8x^5-xy$ 的最小值, $x \in [0,100]$

```
while( T > Tmin )
{
    for (int i = 0; i < k; i++)
    {
        y_current = F(x[i]); // x[i]中存在初始化的值
        x_new = x[i] + random();
        if (x_new >= 0 && x_new <= 100)
        {
            y_new = F(x_new);
            if (y_new < y_current)
                x[i] = x_new;
            else
            {

$$p = \frac{1}{1 + e^{-(y_{new} - y_{current})/T}};$$

                if (random() > p)
                    x[i] = x_new;
            }
        }
    }
    T = T * delta; // delta < 1, 温度下降速率
}
for (i = 0; i < k; i++) result = min(result, F(x[i]));
```



玻尔兹曼机

□ 玻尔兹曼机使用模拟退火算法训练的步骤如下所示：

- 初始化权重 w_{ij} 和阈值 b_j ；
- 从 n 个神经元中随机地选取神经元 j ；
- 计算神经元 j 的输入： $net_j = \sum_i w_{ij}x_i - b_j$ ；
- 以概率 P_j 来决定神经元 j 的输出： $P_j(1) = \frac{1}{1+e^{-net_j/T}}$ ；
- 其他神经元的状态保持不变，即 $\forall i \neq j \quad v_i(t+1) = v_i(t)$ ；
- 根据输出 x_i 和 x_j 的值，调整连接权重 w_{ij} 和阈值 b_j ；
- 返回步骤1)，不断迭代，直到网络不发生变化或满足终止条件。



玻尔兹曼机

□ 调整连接权重 w_{ij} 和偏置 b_i , 这里用似然函数 $L(\theta)$ 导出调整值,
 θ 表示所有的连接权重和偏置

$$L(\theta) = \prod_{n=1}^N p(x_n|\theta)$$

– 其中, 概率分布的定义如下, E 表示能量函数, $Z(\theta)$ 是归一化常数

$$p(x_n|\theta) = \frac{1}{Z(\theta)} \exp\{-E(x_n, \theta)\}$$

$$Z(\theta) = \sum_x \exp\{-E(x, \theta)\}$$



玻尔兹曼机

□ 通常，使用对数似然函数求解

$$\ln L(\theta) = \sum_{n=1}^N \ln p(x_n|\theta)$$

- 当对数似然函数的梯度为0时，就可以得到最大似然估计量，即通过求连接权重 w_{ij} 和偏置 b_i 的相关梯度，可以求出调整值



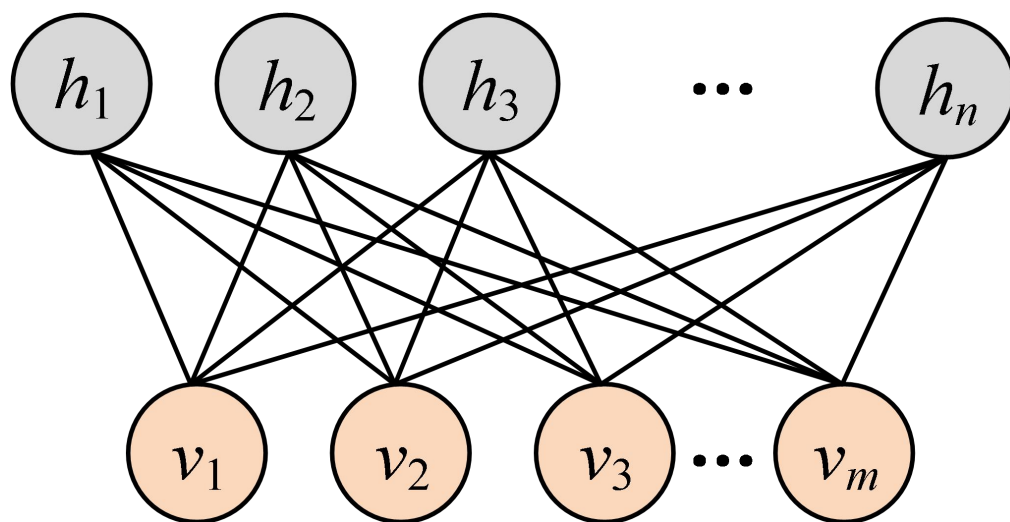
受限玻尔兹曼机

- ❑ 受限玻尔兹曼机（Restricted Boltzmann Machine, RBM）是玻尔兹曼机的受限版本，由Ruslan Salakhutdinov等人提出。
- ❑ 玻尔兹曼机中的神经元之间具有双向连接，这种模型的特点是可以基于模型的抽样从未知的概率分布中学习样本的重要特征。
- ❑ 然而，这种学习的过程是非常困难和耗时的，因此在求解实际问题时使用的并不多，RBM的提出就是通过玻尔兹曼机网络拓扑结构上加上一些限制来缓解这一问题。

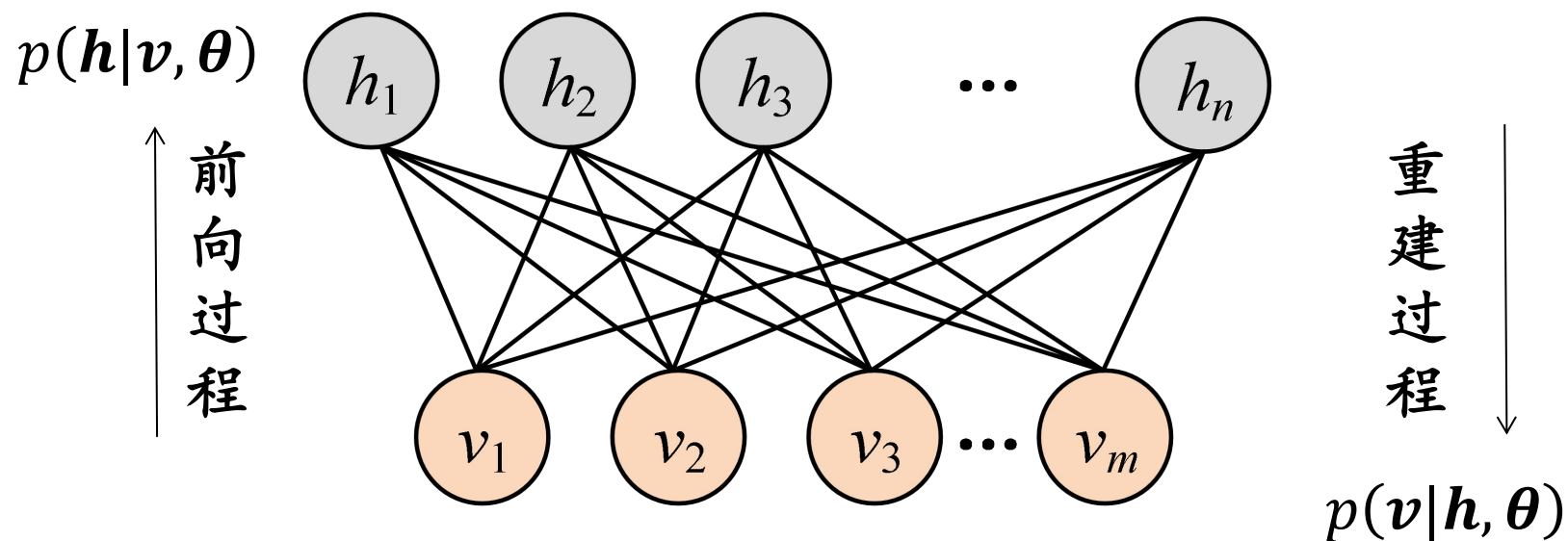


受限玻尔兹曼机

□ 具体来说，RBM是一个马尔可夫随机场（Markov Random Field, MRF），也是一个双边无向图模型，如图所示。图中有 m 个可见神经元 $\mathbf{v}=(v_1, v_2, \dots, v_m)$ ，表示可见数据，还有 n 个隐藏神经元 $\mathbf{h}=(h_1, h_2, \dots, h_n)$ ，用来表示可见变量之间的依赖。



受限玻尔兹曼机



受限玻尔兹曼机

□ 二元RBM(v, h)随机变量值的值域为 $\{0, 1\}$ 。模型(v, h)的概率分布为：

$$p(v, h) = \frac{1}{Z} e^{-E(v, h)}$$

□ 那么可见变量 v 的概率分布即为联合分布的边缘分布：

$$p(v) = \sum_h p(v, h) = \frac{1}{Z} \sum_h e^{-E(v, h)}$$



受限玻尔兹曼机

□ 其中， $Z = \sum_{v,h} e^{-E(v,h)}$ ，称作配分函数(partition function)，其计算是非常困难的。 $E(v,h)$ 称作能量函数，计算公式为：

$$E(v, h) = - \sum_{i=1}^m \sum_{j=1}^n w_{ij} v_i h_j - \sum_{i=1}^m b_i v_i - \sum_{j=1}^n c_j h_j$$

- 其中， w_{ij} 是神经元 v_i 和 h_j 之间的实值权重， b_i 和 c_j 分别是第 i 个可见神经元和第 j 个隐藏神经元上的实值偏置项。



受限玻尔兹曼机

- 假定训练样本集合为 $S=(v^1, v^2, \dots, v^{n_s})$, 其中 n_s 为训练样本的数目, $v^i = (v_1^i, v_2^i, \dots, v_{n_v}^i)^T$, $i=1, 2, \dots, n_s$, 它们是独立同分布的, 设RBM的参数集合为 θ , 则训练RBM的目标就是最大化如下似然:

$$L_{\theta, S} = \prod_{i=1}^{n_s} p(v^i | \theta)$$

- 一般通过对数转化为连加的形式, 其等价形式

$$\log L_{\theta, S} = \sum_{i=1}^{n_s} \log p(v^i | \theta)$$

- θ 包括 w_{ij} 、 b_i 和 c_j , 其中 $i=1, 2, \dots, m$, $j=1, 2, \dots, n$, 给定一个单独的训练样本 v 时, $\log$ 似然函数表示为:

$$\log p(v | \theta) = \log \left(\frac{1}{Z} \sum_h e^{-E(v, h, \theta)} \right)$$



受限玻尔兹曼机

□ 对数域的表达:

$$\begin{aligned}\log p(\mathbf{v}|\boldsymbol{\theta}) &= \log\left(\frac{1}{Z} \sum_h e^{-E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta})}\right) \\ &= \log\left(\sum_h e^{-E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta})}\right) - \log Z \\ &= \log\left(\sum_h e^{-E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta})}\right) - \log \sum_{\mathbf{v}, \mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta})}\end{aligned}$$



受限玻尔兹曼机

□ 计算对参数 θ 的梯度，得到：

$$\frac{\partial \log p(\mathbf{v}|\theta)}{\partial \theta} = \frac{\partial (\log(\sum_h e^{-E(\mathbf{v}, \mathbf{h}, \theta)}))}{\partial \theta} - \frac{\partial (\log \sum_{\mathbf{v}, \mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h}, \theta)})}{\partial \theta}$$

$$\frac{\partial \log p(\mathbf{v}|\theta)}{\partial \theta} = -\frac{1}{\sum_h e^{-E(\mathbf{v}, \mathbf{h}, \theta)}} \sum_h e^{-E(\mathbf{v}, \mathbf{h}, \theta)} \frac{\partial (E(\mathbf{v}, \mathbf{h}, \theta))}{\partial \theta} + \frac{1}{\sum_{\mathbf{v}, \mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h}, \theta)}} \sum_{\mathbf{v}, \mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h}, \theta)} \frac{\partial (E(\mathbf{v}, \mathbf{h}, \theta))}{\partial \theta}$$

$$\frac{e^{-E(\mathbf{v}, \mathbf{h}, \theta)}}{\sum_h e^{-E(\mathbf{v}, \mathbf{h}, \theta)}} = \frac{e^{-E(\mathbf{v}, \mathbf{h}, \theta)} / Z}{\sum_h e^{-E(\mathbf{v}, \mathbf{h}, \theta)} / Z} = \frac{p(\mathbf{v}, \mathbf{h}|\theta)}{p(\mathbf{v}|\theta)} = p(\mathbf{h}|\mathbf{v}, \theta)$$

$$\frac{\partial \log p(\mathbf{v}|\theta)}{\partial \theta} = -\sum_h p(\mathbf{h}|\mathbf{v}, \theta) \frac{\partial (E(\mathbf{v}, \mathbf{h}, \theta))}{\partial \theta} + \sum_{\mathbf{v}, \mathbf{h}} p(\mathbf{v}, \mathbf{h}, \theta) \frac{\partial (E(\mathbf{v}, \mathbf{h}, \theta))}{\partial \theta}$$



受限玻尔兹曼机

□ 进一步分解：

$$\begin{aligned}\frac{\partial \log p(\mathbf{v}|\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} &= - \sum_h p(\mathbf{h}|\mathbf{v}, \boldsymbol{\theta}) \frac{\partial (E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}))}{\partial \boldsymbol{\theta}} + \sum_{\mathbf{v}, \mathbf{h}} p(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \frac{\partial (E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}))}{\partial \boldsymbol{\theta}} \\ \sum_{\mathbf{v}, \mathbf{h}} p(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \frac{\partial (E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}))}{\partial \boldsymbol{\theta}} &= \sum_{\mathbf{v}} p(\mathbf{v}|\boldsymbol{\theta}) \sum_h p(\mathbf{h}|\mathbf{v}, \boldsymbol{\theta}) \frac{\partial (E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}))}{\partial \boldsymbol{\theta}}\end{aligned}$$

□ 因此，计算 $\sum_h p(\mathbf{h}|\mathbf{v}, \boldsymbol{\theta}) \frac{\partial (E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}))}{\partial \boldsymbol{\theta}}$



受限玻尔兹曼机

□ θ 为 w_{ij} 时,

$$\begin{aligned}\sum_{\mathbf{h}} p(\mathbf{h}|\mathbf{v}, \boldsymbol{\theta}) \frac{\partial(E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}))}{\partial w_{ij}} &= - \sum_{\mathbf{h}} p(\mathbf{h}|\mathbf{v}, \boldsymbol{\theta}) h_j v_i \\&= - \sum_{\mathbf{h}} \prod_{k=1}^{n_k} p(h_k|\mathbf{v}, \boldsymbol{\theta}) h_j v_i \\&= - \sum_{\mathbf{h}} p(h_i|\mathbf{v}, \boldsymbol{\theta}) p(h_{-i}|\mathbf{v}, \boldsymbol{\theta}) h_j v_i \\&= - \sum_{h_i} \sum_{h_{-i}} p(h_i|\mathbf{v}, \boldsymbol{\theta}) p(h_{-i}|\mathbf{v}, \boldsymbol{\theta}) h_j v_i \\&= - \sum_{h_i} p(h_i|\mathbf{v}, \boldsymbol{\theta}) h_j v_i \sum_{h_{-i}} p(h_{-i}|\mathbf{v}, \boldsymbol{\theta}) \\&= - \sum_{h_i} p(h_i|\mathbf{v}, \boldsymbol{\theta}) h_j v_i \\&= - (p(h_i = 0|\mathbf{v}, \boldsymbol{\theta}) \cdot 0 \cdot v_i + p(h_i = 1|\mathbf{v}, \boldsymbol{\theta}) \cdot 1 \cdot v_i)\end{aligned}$$

在RBM中，只有不同层的变量之间有连接，而同一层的变量之间无连接，所以给定可见变量、隐藏变量是条件独立的，反过来也成立，即：

$$p(\mathbf{h}|\mathbf{v}) = \prod_{j=1}^n p(h_j|\mathbf{v}), p(\mathbf{v}|\mathbf{h}) = \prod_{i=1}^m p(v_i|\mathbf{h})$$



受限玻尔兹曼机

□ θ 为 w_{ij} 时,

$$\sum_h p(\mathbf{h}|\mathbf{v}, \boldsymbol{\theta}) \frac{\partial(E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}))}{\partial w_{ij}} = -p(h_i = 1|\mathbf{v}, \boldsymbol{\theta})v_i$$

$$\begin{aligned} \frac{\partial \log p(\mathbf{v}|\boldsymbol{\theta})}{\partial w_{ij}} &= -\sum_h p(\mathbf{h}|\mathbf{v}, \boldsymbol{\theta}) \frac{\partial(E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}))}{\partial \theta} + \sum_{\mathbf{v}, \mathbf{h}} p(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \frac{\partial(E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}))}{\partial \theta} \\ &= -p(h_i = 1|\mathbf{v}, \boldsymbol{\theta})v_i + -p(h_i = 1|\mathbf{v}, \boldsymbol{\theta})v_i \end{aligned}$$



受限玻尔兹曼机

□ 同样可以计算对参数 b_i 和 c_j 的梯度，从而得到三个参数的更新公式如下：

$$\left. \begin{aligned} \Delta w_{ij} &= \frac{\partial \log p(\mathbf{v}|\boldsymbol{\theta})}{\partial w_{ij}} = p(h_j = 1|\mathbf{v})v_i - \sum_{\mathbf{v}} p(\mathbf{v})p(h_j = 1|\mathbf{v})v_i \\ \Delta b_i &= \frac{\partial \log p(\mathbf{v}|\boldsymbol{\theta})}{\partial b_i} = v_i - \sum_{\mathbf{v}} p(\mathbf{v})v_i \\ \Delta c_j &= \frac{\partial \log p(\mathbf{v}|\boldsymbol{\theta})}{\partial c_j} = p(h_j = 1|\mathbf{v}) - \sum_{\mathbf{v}} p(\mathbf{v})p(h_j = 1|\mathbf{v}) \end{aligned} \right\}$$



受限玻尔兹曼机

- ❑ RBM训练采用log似然梯度上升法，如前面计算对参数 θ 的梯度的公式所示，梯度计算中主要包括两项：第一项叫做正阶段项，是当可见变量被训练数据赋值时，对隐藏变量抽样的结果；第二项叫做负阶段项，计算这一项要求获得模型的联合样本。
- ❑ 正阶段项是非常容易获得的，然而，负阶段项却是非常难以计算的，这是因为需要长时间运行吉布斯链才能使网络收敛到稳态。
- ❑ 因此，为了训练的高效性，现存的RBM训练算法大多是对负阶段项进行估计。对比散度算法（Contrast divergence, CD）是RBM训练常用的近似算法。



受限玻尔兹曼机

k 步对比散度算法

输入: $\text{RBM}(v_1, \dots, v_m, h_1, \dots, h_n)$, 训练批数据 $\mathbf{S}$

输出: 梯度估计值 $\Delta\omega_{ij}, \Delta b_j, \Delta c_i$, 其中 $i = 1, 2, \dots, m, j = 1, 2, \dots, n$

初始化: $\Delta\omega_{ij} = \Delta b_j = \Delta c_i = 0$, 其中 $i = 1, 2, \dots, m, j = 1, 2, \dots, n$

for $\mathbf{S}$ 中所有的训练样本 $\mathbf{v}$ **do**

$\mathbf{v}^{(0)} \leftarrow \mathbf{v}$

for $t = 0, \dots, k - 1$ **do**

for $j = 1, \dots, n$ **do** 采样 $h_i^{(t)} \sim p(h_i | \mathbf{v}^{(t)})$

for $i = 1, \dots, m$ **do** 采样 $v_j^{(t+1)} \sim p(v_j | \mathbf{h}^{(t)})$

for $i = 1, \dots, m, j = 1, \dots, n$ **do**

$\Delta\omega_{ij} \leftarrow \Delta\omega_{ij} + p(h_j = 1 | \mathbf{v}^{(0)})v_i^{(0)} - p(h_j = 1 | \mathbf{v}^{(k)})v_i^{(k)}$

$\Delta b_i \leftarrow \Delta b_i + v_i^{(0)} - v_i^{(k)}$

$\Delta c_j \leftarrow \Delta c_j + p(h_j = 1 | \mathbf{v}^{(0)}) - p(h_j = 1 | \mathbf{v}^{(k)})$





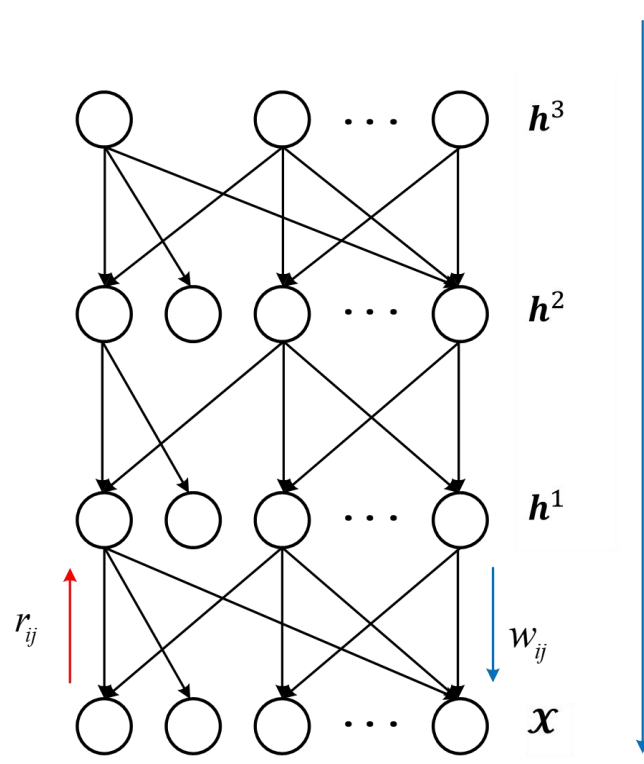
4

深度信念网络与深度玻尔兹曼机



Sigmoid信念网

- ❑ 1990年，Radford M. Neal提出了Sigmoid信念网，它是一种生成式多层神经网络，如图所示。
- ❑ 图中可见数据是 x ， k 层的隐含因子是向量 h^k ，最顶层 h^3 具有可因式分解的先验概率。
- ❑ 在Sigmoid信念网中，每一层中的神经元之间都是相互独立的，典型的神经元是二元随机变量。



Sigmoid信念网

- Sigmoid信念网中的条件分布不同于传统神经网络，计算方式是从上至下而不是从下至上的，这种基于条件分布的典型参数化分布公式为：

$$P(h_i^k = 1 | \mathbf{h}^{k+1}) = \text{sigmoid} \left(b_i^k + \sum_j w_{i,j}^{k+1} h_j^{k+1} \right)$$

- 其中， h_i^k 表示第 k 层中隐藏节点 i 的二元激活， $\mathbf{h}^{k+1}$ 是一个向量 $(h_1^{k+1}, h_2^{k+1}, \dots)$ ，输入向量 $\mathbf{x} = \mathbf{h}^0$ 。 b_i^k 表示第 k 层节点 i 的偏置， $w_{i,j}^{k+1}$ 表示第 $(k+1)$ 层节点 i 与第 k 层节点 j 之间的权重， h_j^{k+1} 表示第 $(k+1)$ 层节点 j 的变量值， $P(\cdot)$ 表示针对模型的概率分布，而不是针对训练样本的概率分布。最底层生成一个输入空间的向量 $\mathbf{x}$ ，模型在训练数据上的概率越高，说明生成的向量与样本越相似，也说明模型能很好的表示这些训练数据。



Sigmoid信念网

□ 考虑到模型具有多层，生成模型分解为：

$$P(x, h^1, \dots, h^l) = P(h^l) \prod_{k=1}^{l-1} P(h^k | h^{k+1}) P(x | h^1)$$

□ 将 $P(x, h^1, h^2, \dots, h^l)$ 进行边缘化即得到 $P(x)$ ，但是在实际中这种计算非常难处理。

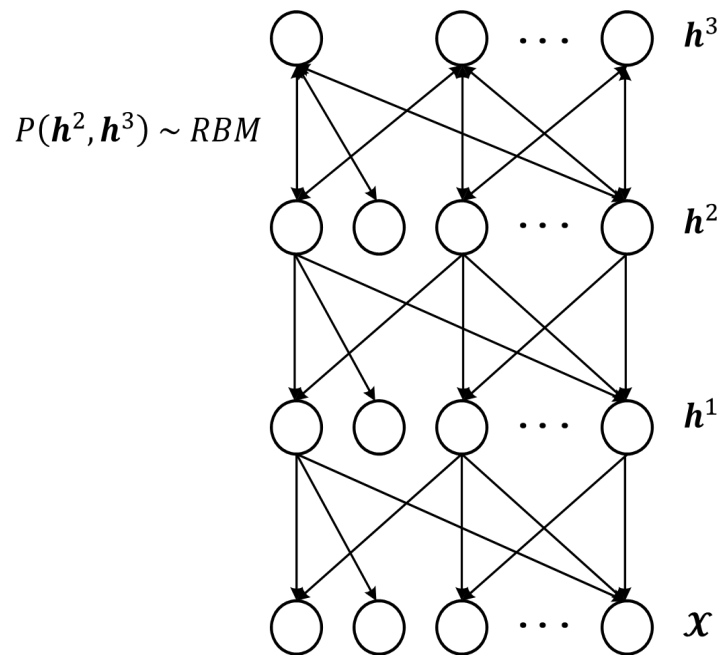
□ Sigmoid信念网一般采用Wake-Sleep算法进行训练：

- Wake阶段：固定红色箭头所指方向的权重，如 r_{ij} ，自底向上生成样本，然后反向传播学习蓝色箭头所指方向的权重，如 w_{ij} ；
- Sleep阶段：固定蓝色箭头所指方向的权重，如 w_{ij} ，自顶向下生成样本，然后反向传播学习红色箭头所指方向的权重，如 r_{ij} ；
- 反复迭代，最后找到满足条件的权重值。



深度信念网

- ❑ 深度信念网络（Deep Belief Networks, DBN）与Sigmoid信念网相似，但是最上面两层有些不同，如图所示。
- ❑ 图中所示为一个具有可见层 x 和隐层 h^1 、 h^2 和 h^3 的DBN。Sigmoid信念网的最顶层为可因式分解的先验概率 $P(h^3)$ ，而图中DBN中最上面两层是一个受限玻尔兹曼机 $P(h^2, h^3)$



深度信念网

□ RBM是一个无向图模型，因此整个DBN模型是一个混合模型，包含 l 个隐藏层的DBN的联合概率为：

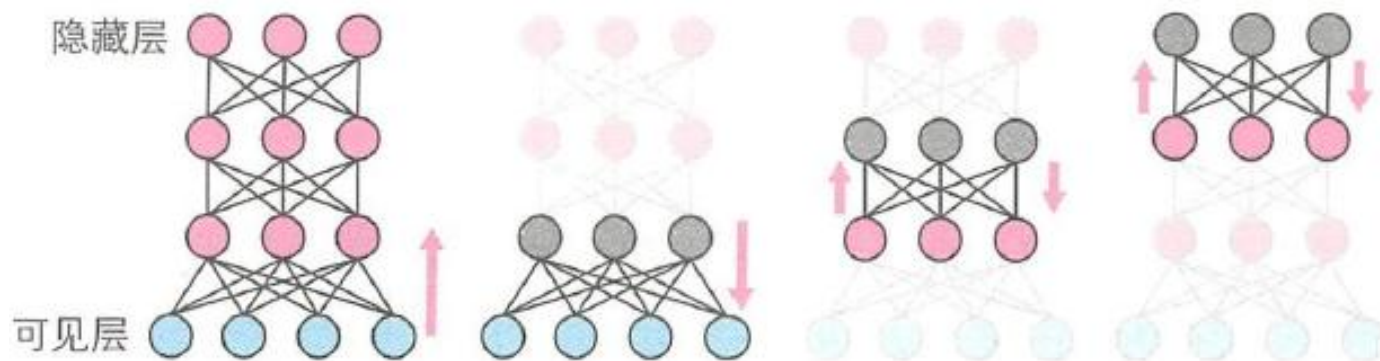
$$P(x, \mathbf{h}^1, \dots, \mathbf{h}^l) = P(\mathbf{h}^{l-1}, \mathbf{h}^l) \prod_{k=1}^{l-2} P(\mathbf{h}^k | \mathbf{h}^{k+1}) P(x | \mathbf{h}^1)$$

– 其中，最上面两层的联合分布 $P(\mathbf{h}^{l-1}, \mathbf{h}^l)$ 为RBM的概率分布。



深度信念网

- DBN的训练主要分为预训练和参数调优两部分，预训练采用贪心逐层堆叠RBM的方式进行，而参数调优一般采用有监督学习算法。

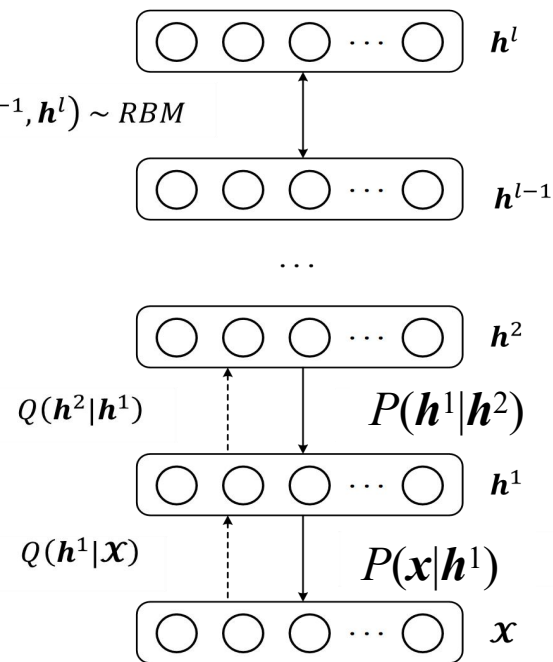


深度信念网

❑ 如图所示是一个具有 l 个隐层的DBN，生成模型的路径用实线箭头上的 P 分布表示，用于生成样本，包括条件分布 $P(h^k|h^{k+1})$ 和顶层联合分布 $P(h^{l-1}, h^l)$ ，而识别路径用虚线箭头上的 Q 分布表示，用于训练和推理。

❑ 最上面两层 h^{l-1} 和 h^l 构成一个RBM，下面各层构成一个有向图模型，即Sigmoid信念网。

❑ $Q(h^{k+1}|h^k)$ 用来估计 $P(h^k|h^{k+1})$ ，但是最顶层 $Q(h^l|h^{l-1})$ 与真实的 $P(h^{l-1}|h^l)$ 是相等的，因为 $P(h^{l-1}, h^l)$ 表示的是RBM，而RBM可以进行准确的推理。



深度信念网

- 在DBN训练过程中，每一层都当作RBM来训练， $Q(h^k, h^{k-1})$ 表示用这种方式训练的第 k 个RBM，而 $P(.)$ 表示DBN中的概率。
- 由于 $Q(h^k|h^{k-1})$ 是可因式分解的，所以计算起来比较容易，而 $P(h^k|h^{k-1})$ 是非因式分解的，计算非常复杂，所以使用 $Q(h^k|h^{k-1})$ 作为 $P(h^k|h^{k-1})$ 的近似值。
- 为了得到DBN中每层的近似后验概率，执行如下步骤：（1）从第一层RBM中抽样 $h^1 \sim Q(h^1|x)$ ；（2）将样本 h^1 作为第二层RBM的输入，计算 h^2 ；（3）不断重复上述过程，直到最后一层。
- DBN训练的伪代码如下：



深度信念网

DBN训练算法：完全无监督预训练DBN，贪婪逐层堆叠RBM

输入：DBN网络结构 $(\mathbf{x}, \mathbf{h}^1, \mathbf{h}^2, \dots, \mathbf{h}^l)$ ，训练样本 $\mathbf{S}$ ，大小为 M

输出：权重 $\mathbf{w}^1, \mathbf{w}^2, \dots, \mathbf{w}^l$ ，偏置 $\mathbf{b}^1, \mathbf{b}^2, \dots, \mathbf{b}^l$ 和 $\mathbf{c}^1, \mathbf{c}^2, \dots, \mathbf{c}^l$

初始化： $\Delta\omega_{ij} = \Delta b_j = \Delta c_i = 0$ ，其中 $i = 1, 2, \dots, n$ ， $j = 1, 2, \dots, m$

for 所有 $\mathbf{S}$ 中的训练样本 $\mathbf{v}^{(i)}$ **do**

 //训练第一个RBM

 将 $\mathbf{x}$ 初始化为 $\mathbf{v}^{(i)}$

$$\mathbf{h}^0 = \mathbf{x}$$

 调用 $RBM(\mathbf{h}^0, \mathbf{h}^1)$ ，得到 $\mathbf{w}^1, \mathbf{b}^1$ 和 $\mathbf{c}^1$

 //逐层堆叠RBM

for $k = 1: l - 1$ **do**

 将 $Q(\mathbf{h}^k | \mathbf{h}^{k-1})$ 作为 $\mathbf{h}^k$ 的初始值

 调用 $RBM(\mathbf{h}^k, \mathbf{h}^{k+1})$ ，得到 $\mathbf{w}^{k+1}, \mathbf{b}^{k+1}$ 和 $\mathbf{c}^{k+1}$

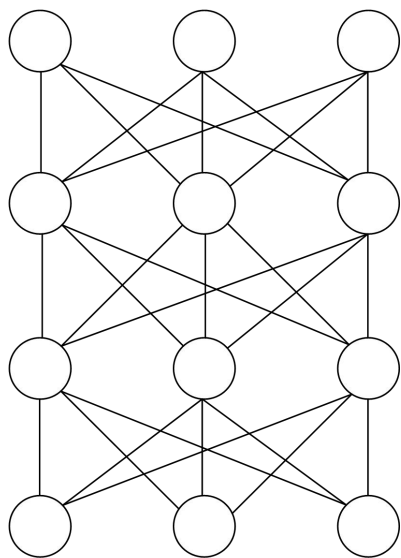
end

end

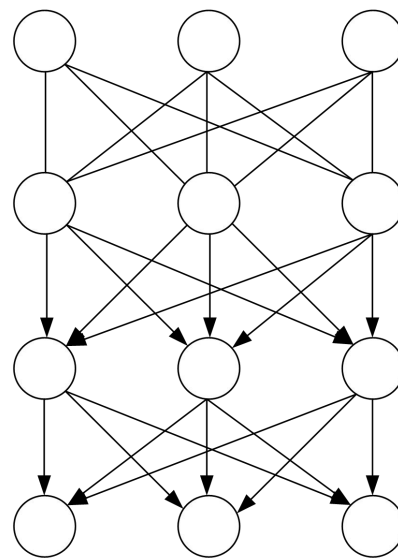


深度玻尔兹曼机

- ❑ 深度玻尔兹曼机（Deep Boltzmann Machine, DBM）是由 Ruslan Salakhutdinov 和 Geffery Hinton 于 2009 年提出来的
- ❑ 它和深度信念网络有很多相似之处，都含有受限玻尔兹曼机，但也有很大的差别，如图所示。



(a) 深度玻尔兹曼机



(b) 深度信念网络

深度玻尔兹曼机

- 从图中可以看出，深度信念网络除了最顶层是无向的之外，其余层都是有向的，而深度玻尔兹曼自底向上都是无向的，这也决定了它们的训练方法不同。
- 如下图所示，对于一个两层DBM（一个可见层 v ，两个隐藏层 h^1 和 h^2 ），其能量函数：

$$E(v, h^1, h^2; \theta) = -v^T w^1 h^1 - h^{1T} w^2 h^2$$

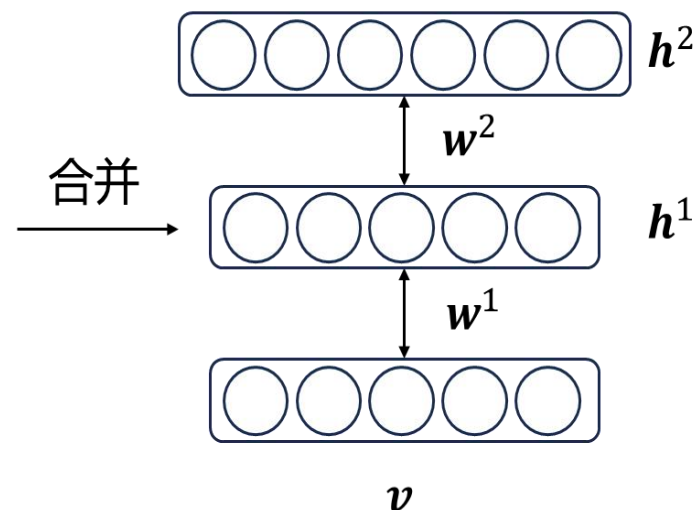
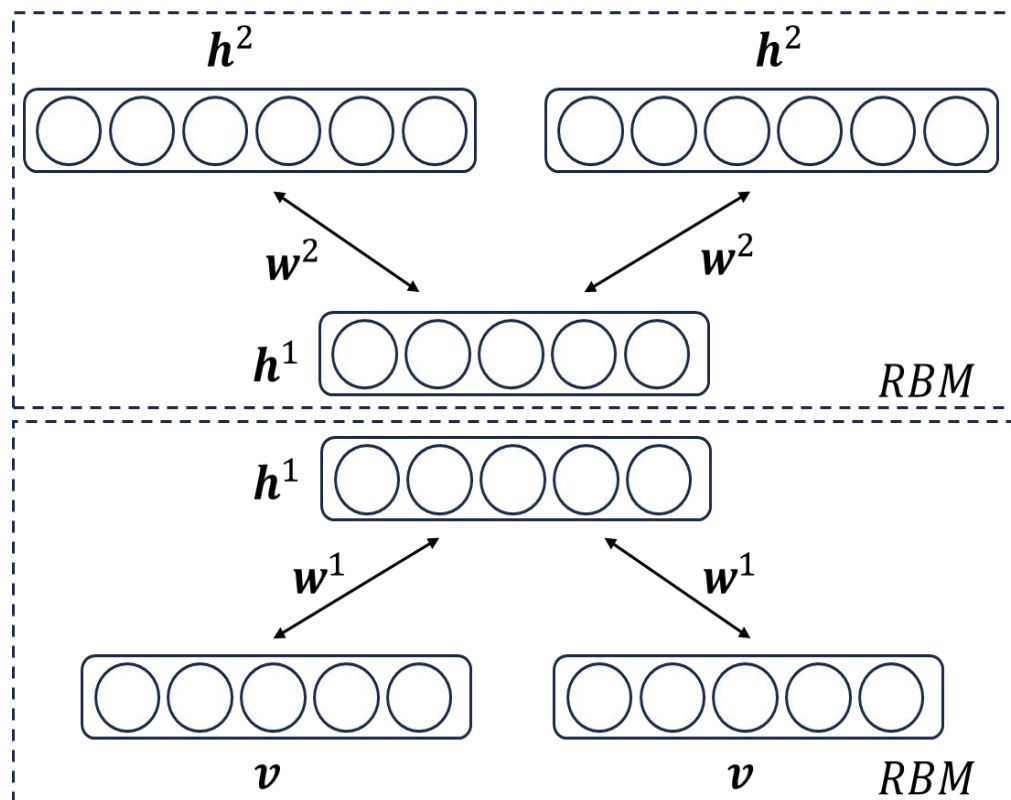
- 其中 w^1 和 w^2 分别表示输入层到第一个隐藏层、第一个隐藏层到第二个隐藏层的参数。可见层的概率密度函数为：

$$p(v; \theta) = \frac{1}{Z(\theta)} \sum_{h^1, h^2} e^{-E(v, h^1, h^2; \theta)}$$



深度玻尔兹曼机

预训练



深度玻尔兹曼机

□ 其中 $Z(\theta)$ 为归一化因子，可见层和两个隐藏层之间的条件分布可由Sigmoid函数给出：

$$p(h_j^1 = 1 | \mathbf{v}, \mathbf{h}^2) = \text{sigmoid}(\sum_i w_{ij}^1 v_i + \sum_m w_{jm}^2 h_m^2)$$

$$p(h_m^2 = 1 | \mathbf{h}^1) = \text{sigmoid}(\sum_j w_{jm}^2 h_j^1)$$

$$p(v_i = 1 | \mathbf{h}^1) = \text{sigmoid}(\sum_j w_{ij}^1 h_j^1)$$



深度玻尔兹曼机

- 深度玻尔兹曼机最常用的训练方式是逐层贪婪预训练加监督微调。逐层贪婪预训练时，每一层被视作独立的RBM进行训练。可见层是对输入数据的建模，之后的每个隐藏层都是对前一RBM后验分布的样本进行建模。
- 当所有的RBM训练完成之后，可以组合成为DBM，用PCD（对比散度算法的一种改进算法）训练即可。
- DBN也是使用逐层贪婪预训练，DBM的训练方法与之类似，但在DBN中，下面几层是单向网络，下一层训练完成之后再训练上一层，而在训练DBM时，要考虑上下两个方向的输入，如上图所示，需要把隐层 h^1 在两个RBM中的产生的值进行合并。





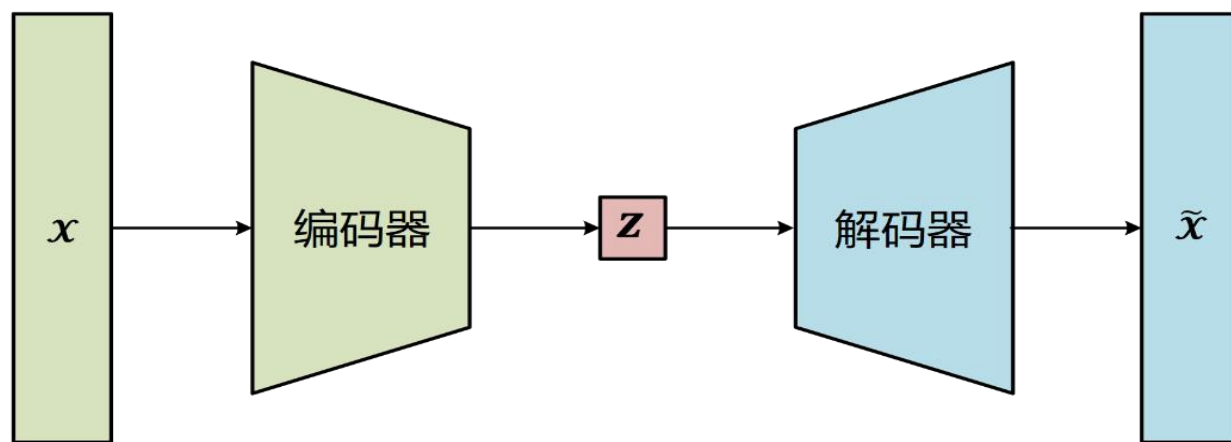
5

自编码器及其变种



自编码器

- ❑ 自编码器的核心思想是使用编码器（Encoder）对高维数据进行降维并表示为一个隐向量，然后将隐向量通过解码器（Decoder）进行重构，使得原始输入与重构的输入尽可能接近。
- ❑ 一个典型的自编码器结构如图所示。其中 x 为输入， $\tilde{x}$ 为输出， z 为隐向量。



自编码器

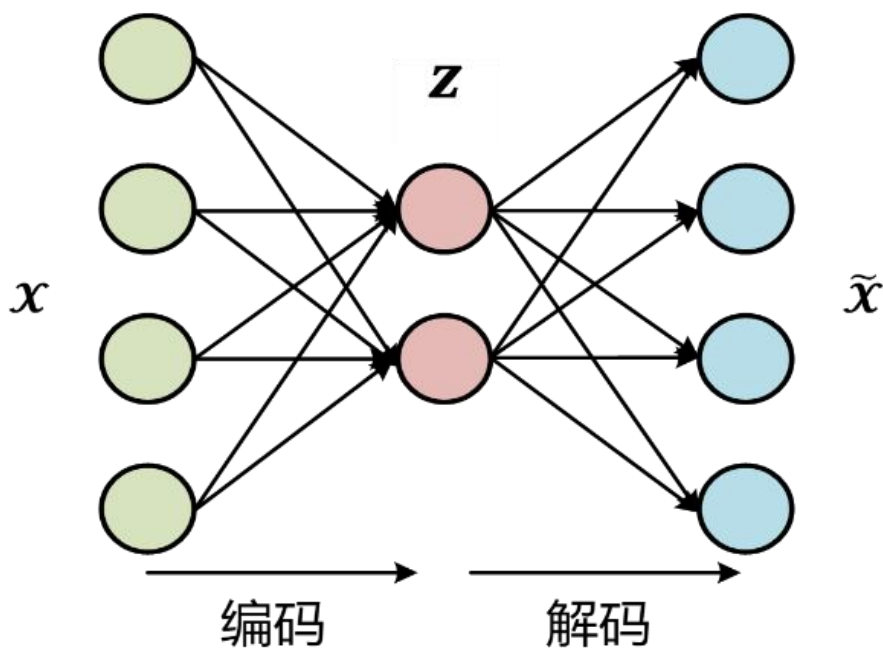
□ 自编码器(Autoencoder): 是一种有效的数据维度压缩算法, 主要应用在以下两个方面:

- 构建一种能够重构输入样本并进行特征表达的神经网络
 - 特征表达: 是指对于分类会发生变动的不稳定模式, 例如手写字体识别中由于不同人的书写习惯和风格的不同造成字符模式不稳定, 或者输入样本中包含噪声等情况, 神经网络也能将其转换成可以准确识别的特征
- 训练多层神经网络时, 通过自编码器训练样本得到参数初始值



自编码器

- 早期的自编码器通常采用一个层数大于或等于三层的前馈神经网络来构建，一般称作前馈自编码器（Feed Forward Autoencoder, FFA）



自编码器

- 前馈自编码器将输入 $x = (x_1, x_2, \dots, x_n)$ 编码为一个新的隐向量 $z = (z_1, z_2, \dots, z_m)$, 要求 $m < n$, 然后再将 z 解码为与输入 x 无限接近的 $\tilde{x} = (\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_n)$
- 用 $f(\cdot)$ 和 $g(\cdot)$ 分别表示编码器与解码器的激活函数, w_1 、 b_1 与 w_2 、 b_2 分别是编码器与解码器的权重向量与偏置向量, 那么有:

$$z = f(w_1 x + b_1)$$

$$\tilde{x} = g(w_2 z + b_2)$$



自编码器

□ 前馈自编码器训练时也使用反向传播算法进行训练，经训练得到 w_1 、 b_1 与 w_2 、 b_2 。但与传统的前馈神经网络不同的地方在于它采用无监督学习方法进行训练，因此误差函数（损失函数）也不尽相同。前馈自编码器一般采用均方误差损失函数或者交叉熵损失函数。

□ 均方误差损失函数如下：

$$(L_{AE})_{MSE} = \frac{1}{n} \sum_{i=1}^n |x_i - \tilde{x}_i|^2$$

□ 交叉熵损失函数如下：

$$(L_{AE})_{CE} = -\frac{1}{n} \sum_{i=1}^n (x_i \log \tilde{x}_i + (1 - x_i) \log(1 - \tilde{x}_i))$$



降噪自编码器

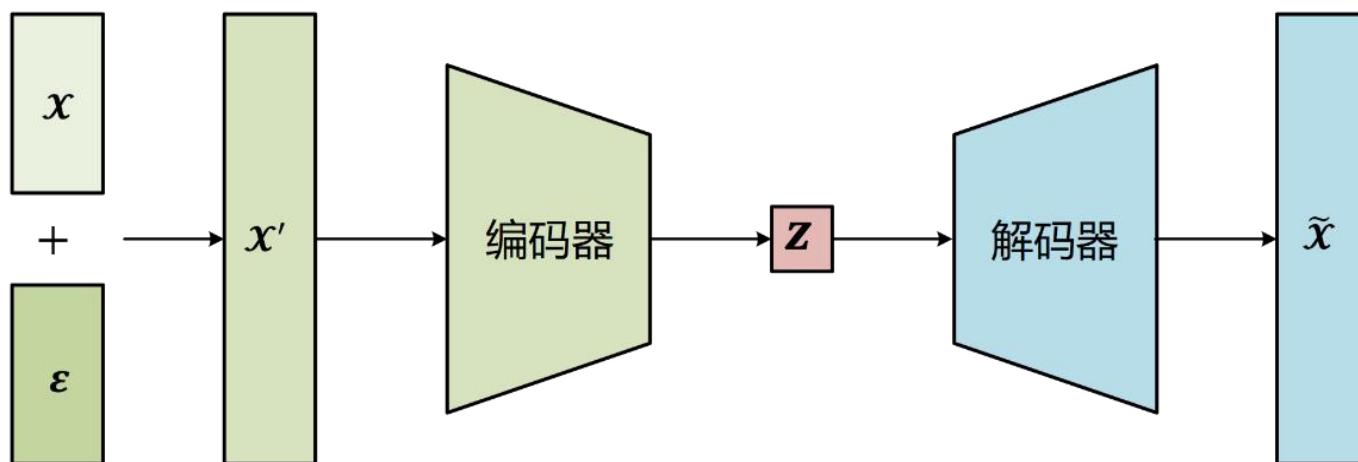
- ❑ 为了防止在原始数据上进行训练时产生过拟合，加入一些噪声来训练自编码器，希望仍然能够重构出原始数据，这就是降噪自编码器。
- ❑ 在训练降噪自编码器之前，首先需要构建噪声数据，通常通过在原始数据 $x = (x_1, x_2, \dots, x_n)$ 中加入一些随机噪声 ε 来构建，常用的构建方式是使用高斯分布：

$$x' \sim \mathcal{N}(x, \sigma^2 I)$$

- ❑ 之后，将加入高斯随机噪声得到的数据 $x' = (x'_1, x'_2, \dots, x'_n)$ 作为降噪自编码器的输入，通过编码为隐向量 $z = (z_1, z_2, \dots, z_m)$ 并解码为输出 $(\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_n)$ ，使之与原始输入 $x = (x_1, x_2, \dots, x_n)$ 尽可能地接近。去噪自编码器的结构如下图所示。



降噪自编码器



稀疏自编码器

- ❑ 自编码器的中间层神经元数量一般都要小于输入层的神经元数量，如果中间层神经元的数量过少，可能会造成一些输入特征的丢失，往往无法重构原始输入，而中间层的神经元数量过多，又可能造成输入特征表达的冗余。
- ❑ 为了解决这一问题，Andrew Ng提出了稀疏自编码器（Sparse Autoencoder, SAE），在自编码器损失函数的基础上引入了正则化项：

$$L_{SAE} = L_{AE} + \Omega(\theta)$$

- ❑ 这使得使用中间层的少数神经元就能够表达输入的压缩表示，并能够重构原始输入。



稀疏自编码器

□ 正则化项采用KL散度来表示，如果损失函数采用均方误差函数，那么稀疏自编码器的损失函数为：

$$(L_{SAE})_{MSE} = \frac{1}{n} \sum_{i=1}^n |x_i - \tilde{x}_i|^2 + \beta \sum_{j=1}^m KL(\rho || \hat{\rho}_j)$$

□ 其中 β 是控制KL散度的权重参数， m 表示中间层神经元的数量， ρ 表示中间层神经元平均激活度的目标值（使得神经元在多数情况下都不激活）， $\hat{\rho}_j$ 表示中间层第 j 个神经元的激活度：

$$\hat{\rho}_j = \frac{1}{n} \sum_{i=1}^n f(w_{ij}x_i + b_j)$$



稀疏自编码器

□ $KL(\rho||\hat{\rho}_j)$ 表示KL散度，它反映了平均激活度和目标值的差异：

$$KL(\rho||\hat{\rho}_j) = \rho \log\left(\frac{\rho}{\hat{\rho}_j}\right) + (1 - \rho) \log\left(\frac{1 - \rho}{1 - \hat{\rho}_j}\right)$$

□ 稀疏自编码器同样采用反向传播算法来进行训练，惩罚项的梯度为：

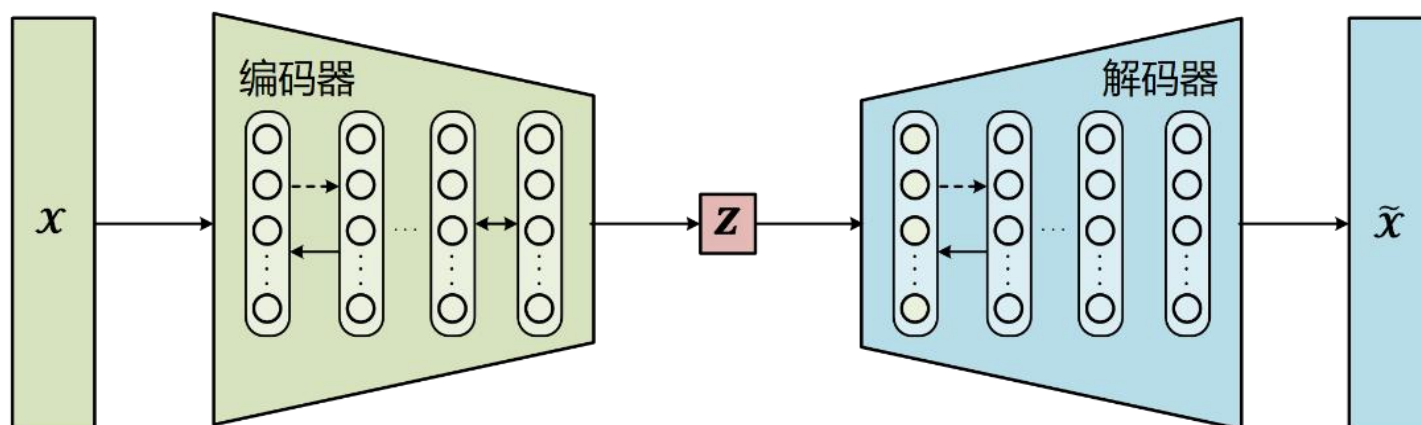
$$\frac{\partial}{\partial \theta} \left(\beta \sum_{j=1}^m KL(\rho||\hat{\rho}_j) \right) = \beta \frac{\partial}{\partial \hat{\rho}_j} KL(\rho||\hat{\rho}_j) \frac{\partial \hat{\rho}_j}{\partial \theta} = \left(-\frac{\rho}{\hat{\rho}_j} + \frac{1 - \rho}{1 - \hat{\rho}_j} \right) \frac{\partial \hat{\rho}_j}{\partial \theta}$$

□ 其中 θ 表示稀疏自编码器要学习的所有参数，如权重和偏置等。



深度自编码器

- 深度自编码器由两个对称的深度信念网络构成，编码器通常由4-5个隐藏层组成，解码器也是由4-5个隐藏层组成，这里的隐藏层一般指的是受限玻尔兹曼机，如图所示。



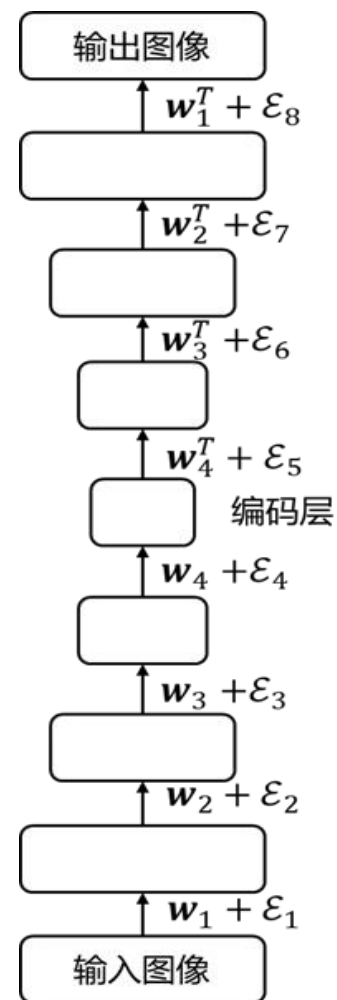
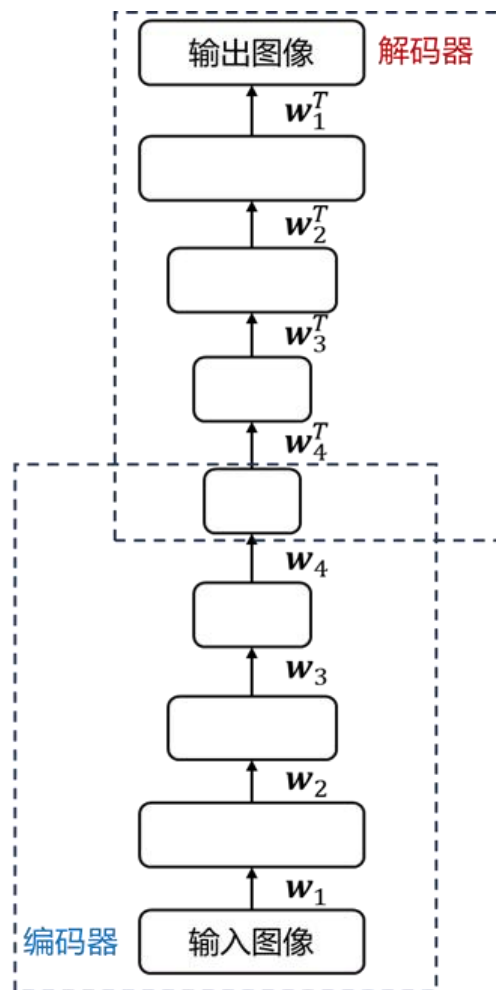
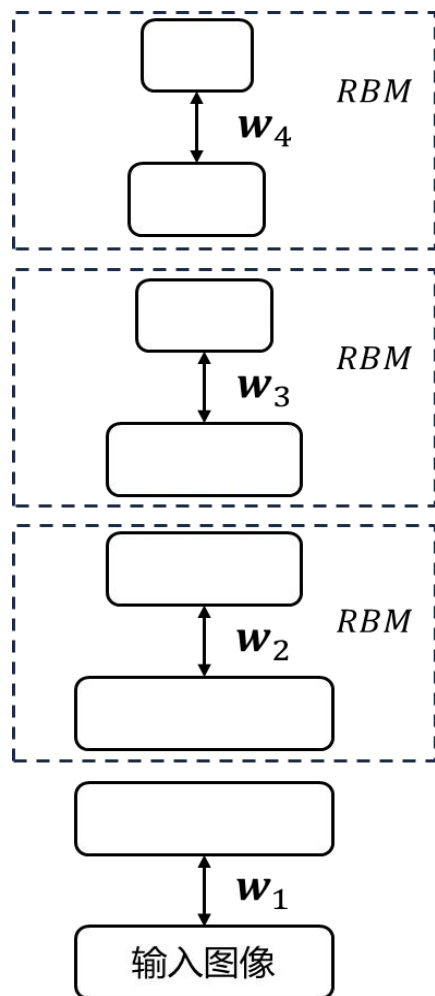
深度自编码器

□ 深度自编码器的训练过程（以图像输入为例）：

- 预训练一个DBN：采用贪心逐层堆叠RBM的方式进行预训练，从而获得各层权重的初始值。
- 使用预训练好的DBN来构造深度自编码器：其中编码器直接使用预训练好的DBN，权重也是预训练好的DBN的权重；解码器将预训练好DBN进行翻转，从而与编码器形成对称结构，解码器的权重是对预训练好的DBN权重进行转置而来。
- 对构造的深度自编码器进行有监督微调：使用反向传播算法对深度自编码器进行微调，从而获得最终的权重值。



深度自编码器





6

扩散模型



扩散模型

□ 扩散模型的相关工作大都起源于OpenAI于2020年提出的降噪扩散概率模型（Denoising Diffusion Probabilistic Models, DDPM）。DDPM包含前向过程（Forward Process）和逆向过程（Reverse Process）

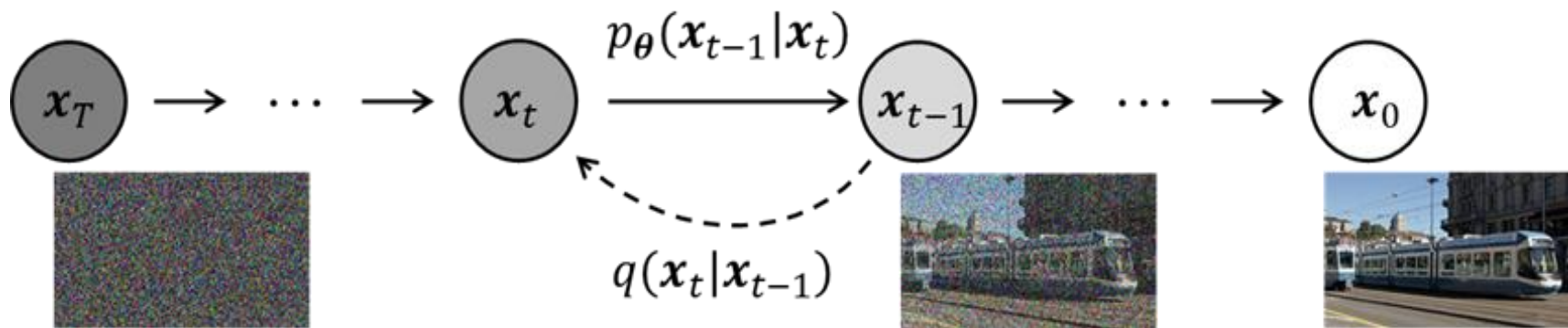
- 前向过程又称为扩散过程（Diffusion Process），本质上是在输入图像数据的基础上逐步注入符合高斯分布的随机噪声，直至图像数据本身变为服从标准高斯分布的随机噪声。
- 而逆向过程，则是进行图像生成的推断过程，当给定一个服从标准高斯分布的噪声，逐步去除噪声从而还原图像。



扩散模型

□ DDPM的前向过程与逆向过程如图所示

- 前向过程如图中的虚线箭头所示的 q 过程，给定真实图像 $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 和 T 个方差超参数 $\boldsymbol{\beta} = \{\beta_t \in (0,1)\}_{t=1}^T$ ，前向过程会逐步在图像中添加高斯噪声，得到图像集合 $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\}$ 。



前向过程

- 此外，每个时刻的状态只与前一时刻的状态相关，因此前向过程是一个马尔可夫过程。对前向过程展开描述如下：
- 以第 t 步为例， $\mathbf{x}_t$ 仅与 $\mathbf{x}_{t-1}$ 相关，那么有：

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \quad (1)$$

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}) \quad (2)$$



前向过程

□ 对于一个高斯分布 $\mathcal{N}(x; \mu_\theta, \sigma_\theta^2 I)$ ，如果要从中采样一个 x ，利用重参数技巧，可以将采样写成：

$$x = \mu_\theta + \sigma_\theta \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (3)$$

□ 因此，对于前向过程中任意步骤的状态 x_t ，可以表示为：

$$\begin{aligned} \bullet \quad x_t &= \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon_{t-1} \\ &= \sqrt{1 - \beta_t} \left(\sqrt{1 - \beta_{t-1}} x_{t-2} + \sqrt{\beta_{t-1}} \epsilon_{t-2} \right) + \sqrt{\beta_t} \epsilon_{t-1} \\ &= \sqrt{(1 - \beta_t)(1 - \beta_{t-1})} x_{t-2} + \left(\sqrt{(1 - \beta_t)\beta_{t-1}} \epsilon_{t-2} + \sqrt{\beta_t} \epsilon_{t-1} \right) \end{aligned} \quad (4)$$

– 其中， $\epsilon_{t-1}, \epsilon_{t-2} \sim \mathcal{N}(0, I)$ 。此外，对于两个独立的高斯分布 $X \sim \mathcal{N}(\mu_X, \sigma_X^2 I)$ 与 $Y \sim \mathcal{N}(\mu_Y, \sigma_Y^2 I)$ ，它们的和仍然服从高斯分布，即 $X + Y \sim \mathcal{N}(\mu_X + \mu_Y, (\sigma_X^2 + \sigma_Y^2) I)$



前向过程

□ 因此上式后一项可写为：

$$\sqrt{(1 - \beta_t)\beta_{t-1}}\epsilon_{t-2} + \sqrt{\beta_t}\epsilon_{t-1} = \sqrt{(1 - \beta_t)\beta_{t-1} + \beta_t}\bar{\epsilon}_{t-2}, \quad \bar{\epsilon}_{t-2} \sim \mathcal{N}(0, I) \quad (5)$$

□ 代入 x_t 中，有：

$$x_t = \sqrt{(1 - \beta_t)(1 - \beta_{t-1})}x_{t-2} + \sqrt{(1 - \beta_t)\beta_{t-1} + \beta_t}\bar{\epsilon}_{t-2}, \quad \bar{\epsilon}_{t-2} \sim \mathcal{N}(0, I) \quad (6)$$

□ 不妨令 $\alpha_t = 1 - \beta_t$

$\bar{\alpha}_t = \prod_{i=1}^T \alpha_i$ ，代入上式，得到：

$$\begin{aligned} x_t &= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{\alpha_t(1 - \alpha_{t-1}) + 1 - \alpha_t}\bar{\epsilon}_{t-2} = \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_t\alpha_{t-1}}\bar{\epsilon}_{t-2} \\ &= \dots = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\bar{\epsilon}_0, \quad \bar{\epsilon}_{t-2}, \dots, \bar{\epsilon}_0 \sim \mathcal{N}(0, I) \end{aligned} \quad (7)$$



前向过程

□ 至此，可以发现在给定方差超参数集合 β 的前提下，任意时刻的 x_t 都可以使用 x_0 和 β 来表示，即：

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I) \quad (8)$$

- 其中， $\alpha_t = 1 - \beta_t$ ， $\bar{\alpha}_t = \prod_{i=1}^T \alpha_i$ ，且当 $T \rightarrow \infty$ ， $\bar{\alpha}_t = \prod_{i=1}^T \alpha_i \rightarrow 0$ ，从而 $x_T \sim \mathcal{N}(0, I)$ ，即当前向过程的步数趋于无穷大时， x_T 最终将变为服从标准高斯分布的随机噪声。



前向过程

□ 如图所示为DDPM前向过程的简单示例，将 T 设置为100，方差超参数 β_t 从0.0001递增到0.1，可以发现当步数增多，原始图像也逐渐变得难以辨认直到完全变为随机噪声。



x_0



x_{10}



x_{20}



x_{40}



x_{70}



x_{100}

逆向过程

- 在前向过程中，通过 $q(x_t|x_{t-1})$ 逐步将真实图像 x_0 变为标准高斯分布噪声 x_T ；反之，如果可以获取前向过程每一步的真实逆向分布 $q(x_{t-1}|x_t)$ ，那也可以从一个标准高斯分布 x_T 逐步去除噪声还原得到原始图像 x_0 。
- 但实际上无法直接对真实逆向分布 $q(x_{t-1}|x_t)$ 进行推断，DDPM使用神经网络 $p_\theta(x_{t-1}|x_t)$ 对逆向分布进行预测。



逆向过程

□ 逆向分布 p_{θ} 可以表示为：

$$p_{\theta}(x_{0:T}) = p_{\theta}(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1}|x_t) \quad (9)$$

$$p_{\theta}(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t)) \quad (10)$$

□ DDPM模型的目标则是学习到正确的 $\mu_{\theta}(x_t, t)$ 和 $\Sigma_{\theta}(x_t, t)$ 预测。虽然无法通过前向过程的分布 $q(x_t|x_{t-1})$ 去简单的推断出真实的逆向分布 $q(x_{t-1}|x_t)$ ，但是可以将 x_0 引入，根据贝叶斯公式，有：

$$\begin{aligned} q(x_{t-1}|x_t, x_0) &= \frac{q(x_{t-1})q(x_0|x_{t-1})q(x_t|x_{t-1}, x_0)}{q(x_0)q(x_t|x_0)} \\ &= \frac{q(x_{t-1}|x_0)q(x_t|x_{t-1}, x_0)}{q(x_t|x_0)} \end{aligned} \quad (11)$$



逆向过程

□ 不妨将 $q(x_{t-1}|x_t, x_0)$ 的均值和方差表示为 $\tilde{\mu}_t(x_t, x_0)$ 和 $\tilde{\beta}_t$, 即:

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I) \quad (12)$$

□ 前面提到前向过程为马尔可夫链, 并且任一时间步的 x_t 都可以使用 x_0 和 β 来表示, 那么可以将公式 (11) 中的每一项分别表示为如下形式:

$$q(x_t|x_{t-1}, x_0) = q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I) \quad (13)$$

$$q(x_{t-1}|x_0) = \mathcal{N}(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}}x_0, (1 - \bar{\alpha}_{t-1})I) \quad (14)$$

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I) \quad (15)$$



逆向过程

□ 对于一元高斯分布 $x \sim \mathcal{N}(\mu, \sigma^2)$ ，其概率密度函数为 $f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$ 。将上述每一分项高斯分布的概率密度函数的指数部分代入，有：

$$\begin{aligned} q(x_{t-1}|x_t, x_0) &= \frac{q(x_{t-1}|x_0)q(x_t|x_{t-1}, x_0)}{q(x_t|x_0)} \\ &\propto \frac{\exp\left(-\frac{(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}}x_0)^2}{2(1 - \bar{\alpha}_{t-1})}\right) \exp\left(-\frac{(x_t - \sqrt{1 - \beta_t}x_{t-1})^2}{2\beta_t}\right)}{\exp\left(-\frac{(x_t - \sqrt{\bar{\alpha}_t}x_0)^2}{2(1 - \bar{\alpha}_t)}\right)} \\ &= \exp\left(-\frac{1}{2}\left(\frac{(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}}x_0)^2}{(1 - \bar{\alpha}_{t-1})} + \frac{(x_t - \sqrt{1 - \beta_t}x_{t-1})^2}{\beta_t} - \frac{(x_t - \sqrt{\bar{\alpha}_t}x_0)^2}{(1 - \bar{\alpha}_t)}\right)\right) \\ &= \exp\left(-\frac{1}{2}\left(\left(\frac{1}{1 - \bar{\alpha}_{t-1}} + \frac{1 - \beta_t}{\beta_t}\right)x_{t-1}^2 - \left(\frac{2\sqrt{\bar{\alpha}_{t-1}}x_0}{1 - \bar{\alpha}_{t-1}} + \frac{2\sqrt{1 - \beta_t}x_t}{\beta_t}\right)x_{t-1} + C(x_t, x_0)\right)\right) \end{aligned} \quad (16)$$



逆向过程

□ 其中， $C(x_t, x_0)$ 为与 x_{t-1} 无关项的组合，可以忽略。此外，一元高斯分布概率密度函数的指数部分可以展开：

$\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) = \exp\left(-\frac{1}{2}\left(\frac{1}{\sigma^2}x^2 - \frac{2\mu}{\sigma^2}x + \frac{\mu^2}{\sigma^2}\right)\right)$ ， $\frac{1}{\sigma^2}$ 和 $\frac{2\mu}{\sigma^2}$ 分别对应公式（16）中的红色和蓝色部分，即： $\frac{1}{\sigma^2}$ 对应红色部分：

$$\frac{1}{\tilde{\beta}_t} = \left(\frac{1}{1 - \bar{\alpha}_{t-1}} + \frac{1 - \beta_t}{\beta_t} \right) = \frac{1 - \bar{\alpha}_t}{(1 - \bar{\alpha}_{t-1})\beta_t} \quad (17)$$

□ $\frac{2\mu}{\sigma^2}$ 对应蓝色部分

$$\frac{2\tilde{\mu}_t(x_t, x_0)}{\tilde{\beta}_t} = \left(\frac{2\sqrt{\bar{\alpha}_{t-1}}x_0}{1 - \bar{\alpha}_{t-1}} + \frac{2\sqrt{1 - \beta_t}x_t}{\beta_t} \right) \quad (18)$$



逆向过程

□ 根据公式(17)和(18)，可以求得 $q(x_{t-1}|x_t, x_0)$ 的均值 $\tilde{\mu}_t(x_t, x_0)$ 和方差 $\tilde{\beta}_t$:

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t x_0 + (1 - \bar{\alpha}_{t-1})\sqrt{\alpha_t}x_t}{1 - \bar{\alpha}_t}, \quad \tilde{\beta}_t = \frac{(1 - \bar{\alpha}_{t-1})\beta_t}{1 - \bar{\alpha}_t} \quad (19)$$

□ 引入前向过程中已得到的 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\bar{\epsilon}_0$, $\bar{\epsilon}_0 \sim \mathcal{N}(0, I)$, 可以得到 $x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1 - \bar{\alpha}_t}\bar{\epsilon}_0)$, 代入公式(19)中的 $\tilde{\mu}_t(x_t, x_0)$, 有:

$$\tilde{\mu}_t(x_t, x_0) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \bar{\epsilon}_0 \right) \quad (20)$$

- 虽然去除了 x_0 的影响, 但是引入了一个新的变量 $\bar{\epsilon}_0$, $\bar{\epsilon}_0$ 在前向过程中为标准高斯分布采样的噪声, 但是在逆向过程中无法得知其真实值, DDPM的做法是引入一个参数化的神经网络模型 $\epsilon_\theta(x_t, t)$ 去预测高斯噪声 $\bar{\epsilon}_0$, θ 表示模型的参数。



逆向过程

□ 至此，已经可以确定DDPM需要学习的逆向过程 $p_{\theta}(x_{t-1}|x_t)$ 的分布：

$$p_{\theta}(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t)) \quad (21)$$

□ 其中：

$$\begin{aligned} \mu_{\theta}(x_t, t) &= \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right), \\ \Sigma_{\theta}(x_t, t) &= \tilde{\beta}_t = \frac{(1 - \bar{\alpha}_{t-1})\beta_t}{1 - \bar{\alpha}_t} \simeq \beta_t \end{aligned} \quad (22)$$

□ DDPM逆向过程可总结为：在给定 x_t 的前提下，首先预测高斯噪声 $\epsilon_{\theta}(x_t, t)$ ，然后计算 $p_{\theta}(x_{t-1}|x_t)$ 的均值 $\mu_{\theta}(x_t, t)$ 和方差 $\Sigma_{\theta}(x_t, t)$ ，最后通过重参数技巧计算得到 x_{t-1} 完成一步推断，循环进行直至得到 x_0 。



DDPM的训练

□ DDPM使用最大似然估计作为优化目标，损失函数为：

$$L = E_{q(x_0)}[-\log p_{\theta}(x_0)] \quad (23)$$

□ 应用变分下限（Variational Lower Bound, VLB）优化负对数似然，在原始损失函数 L 的基础上，引入一项KL散度 $D_{KL}(q(x_{1:T}|x_0)||p_{\theta}(x_{1:T}|x_0))$ ，因为KL散度的值非负，因此满足下列不等式：

$$E_{q(x_0)}[-\log p_{\theta}(x_0)] \leq E_{q(x_0)}[-\log p_{\theta}(x_0) + D_{KL}(q(x_{1:T}|x_0)||p_{\theta}(x_{1:T}|x_0))]$$

— 其中，KL散度的定义如下：

(24)

$$\begin{aligned} D_{KL}(q(x_{1:T}|x_0)||p_{\theta}(x_{1:T}|x_0)) &= \sum \left[q(x_{1:T}|x_0) \cdot \log \frac{q(x_{1:T}|x_0)}{p_{\theta}(x_{1:T}|x_0)} \right] \\ &= E_{q(x_{1:T}|x_0)} \left[\log \frac{q(x_{1:T}|x_0)}{p_{\theta}(x_{1:T}|x_0)} \right] \end{aligned} \quad (25)$$



DDPM的训练

□ 将公式(25)代入到公式 (24)中，并应用贝叶斯公式进行化简，有：

$$\begin{aligned} & -\log p_{\theta}(x_0) \\ & \leq -\log p_{\theta}(x_0) + E_{q(x_{1:T}|x_0)} \left[\log \frac{q(x_{1:T}|x_0)}{p_{\theta}(x_{1:T}|x_0)} \right] \\ & = -\log p_{\theta}(x_0) + E_{q(x_{1:T}|x_0)} \left[\log \underbrace{\frac{q(x_{1:T}|x_0)}{p_{\theta}(x_{0:T})/p_{\theta}(x_0)}}_{\text{由贝叶斯公式}} \right] \\ & = -\log p_{\theta}(x_0) + E_{q(x_{1:T}|x_0)} \left[\log \frac{q(x_{1:T}|x_0)}{p_{\theta}(x_{0:T})} + \log p_{\theta}(x_0) \right] \\ & = E_{q(x_{1:T}|x_0)} \left[\log \frac{q(x_{1:T}|x_0)}{p_{\theta}(x_{0:T})} \right] \\ & E_{q(x_0)} [-\log p_{\theta}(x_0)] \leq E_{q(x_{0:T})} \left[\log \frac{q(x_{1:T}|x_0)}{p_{\theta}(x_{0:T})} \right] = L_{VLB} \end{aligned} \tag{26}$$



DDPM的训练

□ 此时，DDPM的优化目标可以从最小化 L 转换为最小化 L_{VLB} ，由前向过程和逆向过程的介绍可知 $q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})$, $p_{\theta}(\mathbf{x}_{0:T}) = p_{\theta}(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ ，代入公式(26)可进一步推导得出：

$$\begin{aligned} L_{VLB} &= E_{q(\mathbf{x}_{0:T})} \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_{\theta}(\mathbf{x}_{0:T})} \right] \\ &= E_{q(\mathbf{x}_{0:T})} \left[\log \frac{\prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_{\theta}(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)} \right] \\ &= E_{q(\mathbf{x}_{0:T})} \left[-\log p_{\theta}(\mathbf{x}_T) + \sum_{t=1}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)} \right] \end{aligned}$$



DDPM的训练

$$\begin{aligned} &= E_{q(\mathbf{x}_{0:T})} \left[-\log p_{\theta}(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1})}{p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)} + \log \frac{q(\mathbf{x}_1 | \mathbf{x}_0)}{p_{\theta}(\mathbf{x}_0 | \mathbf{x}_1)} \right] \\ &= E_{q(\mathbf{x}_{0:T})} \left[-\log p_{\theta}(\mathbf{x}_T) + \sum_{t=2}^T \log \left(\underbrace{\frac{q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)}{p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)} \frac{q(\mathbf{x}_t | \mathbf{x}_0)}{q(\mathbf{x}_{t-1} | \mathbf{x}_0)}}_{\text{由公式(11)}} \right) + \log \frac{q(\mathbf{x}_1 | \mathbf{x}_0)}{p_{\theta}(\mathbf{x}_0 | \mathbf{x}_1)} \right] \\ &= E_{q(\mathbf{x}_{0:T})} \left[-\log p_{\theta}(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)}{p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t | \mathbf{x}_0)}{q(\mathbf{x}_{t-1} | \mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1 | \mathbf{x}_0)}{p_{\theta}(\mathbf{x}_0 | \mathbf{x}_1)} \right] \\ &= E_{q(\mathbf{x}_{0:T})} \left[-\log p_{\theta}(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)}{p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)} + \log \prod_{t=2}^T \frac{q(\mathbf{x}_t | \mathbf{x}_0)}{q(\mathbf{x}_{t-1} | \mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1 | \mathbf{x}_0)}{p_{\theta}(\mathbf{x}_0 | \mathbf{x}_1)} \right] \\ &= E_{q(\mathbf{x}_{0:T})} \left[-\log p_{\theta}(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)}{p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)} + \log \frac{q(\mathbf{x}_T | \mathbf{x}_0)}{q(\mathbf{x}_1 | \mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1 | \mathbf{x}_0)}{p_{\theta}(\mathbf{x}_0 | \mathbf{x}_1)} \right] \end{aligned}$$



DDPM的训练

$$\begin{aligned} &= E_{q(x_{0:T})} \left[\underbrace{\log \frac{q(x_T|x_0)}{q(x_1|x_0)} \frac{q(x_1|x_0)}{p_\theta(x_0|x_1)} \frac{1}{p_\theta(x_T)}}_{\text{合并对数项}} + \sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_\theta(x_{t-1}|x_t)} \right] \\ &= E_{q(x_{0:T})} \left[\log \frac{q(x_T|x_0)}{p_\theta(x_T)} + \sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_\theta(x_{t-1}|x_t)} - \log p_\theta(x_0|x_1) \right] \\ &= E_{q(x_{0:T})} \left[D_{KL}(q(x_T|x_0)||p_\theta(x_T)) + \sum_{t=2}^T \underbrace{D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t))}_{L_t} - \log p_\theta(x_0|x_1) \right] \end{aligned} \quad (27)$$



DDPM的训练

□ DDPM对上述优化目标进行了简化，只考虑公式 (27) 中红色的 L_t 部分，即计算分布 $q(x_{t-1}|x_t, x_0)$ 与 $p_\theta(x_{t-1}|x_t)$ 之间的KL散度

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I) \quad (28)$$

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \tilde{\beta}_t I) \quad (29)$$

□ 因此：

$$\begin{aligned} L_t &= D_{KL}(q(x_{t-1}|x_t, x_0) || p_\theta(x_{t-1}|x_t)) \\ &= \left[\frac{1}{2\beta_t} ||\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)||^2 \right] \\ &= E_{x_0, \bar{\epsilon}_0} \left[\frac{1}{2\beta_t} || \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \bar{\epsilon}_0 \right) - \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) ||^2 \right] \end{aligned}$$



DDPM的训练

$$\begin{aligned} &= E_{x_0, \bar{\epsilon}_0} \left[\frac{\beta_t^2}{2\beta_t\alpha_t(1-\bar{\alpha}_t)} \|\bar{\epsilon}_0 - \epsilon_\theta(x_t, t)\|^2 \right] \\ &= E_{x_0, \bar{\epsilon}_0} \left[\frac{\beta_t^2}{2\beta_t\alpha_t(1-\bar{\alpha}_t)} \|\bar{\epsilon}_0 - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\bar{\epsilon}_0, t)\|^2 \right] \end{aligned} \quad (30)$$

□ DDPM进一步对 L_t 进行简化，得到：

$$L_{simple} = E_{x_0, \bar{\epsilon}_0} [\|\bar{\epsilon}_0 - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\bar{\epsilon}_0, t)\|^2] \quad (31)$$

□ 可以发现，DDPM最终损失函数的核心就是最小化采样的真实噪声 $\bar{\epsilon}_0$ 与模型所预测噪声 ϵ_θ 之间的均方误差损失。



谢谢！



计算机科学与技术学院

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY